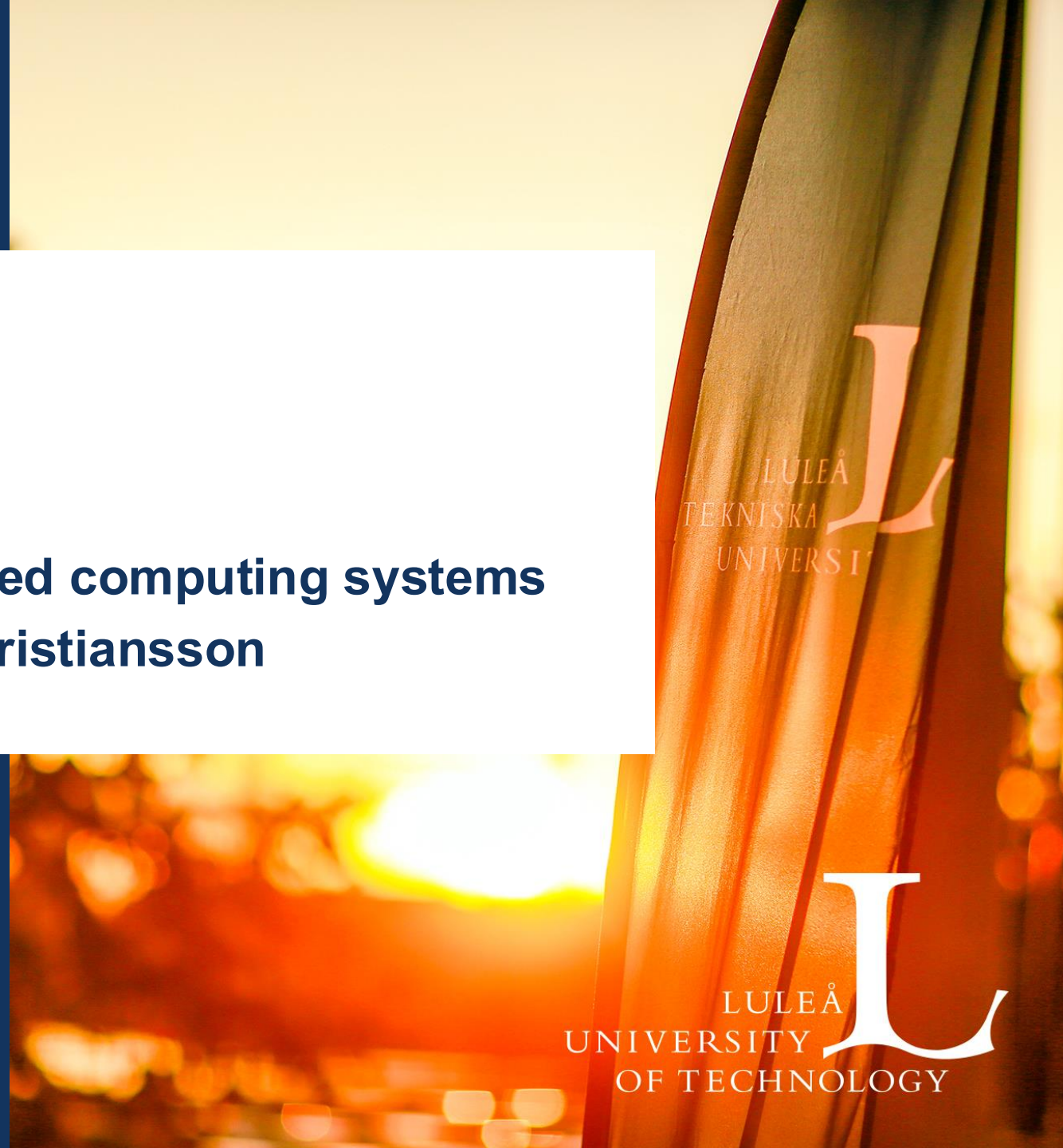
Virtualisation

Mobile and distributed computing systems

Johan Kristiansson



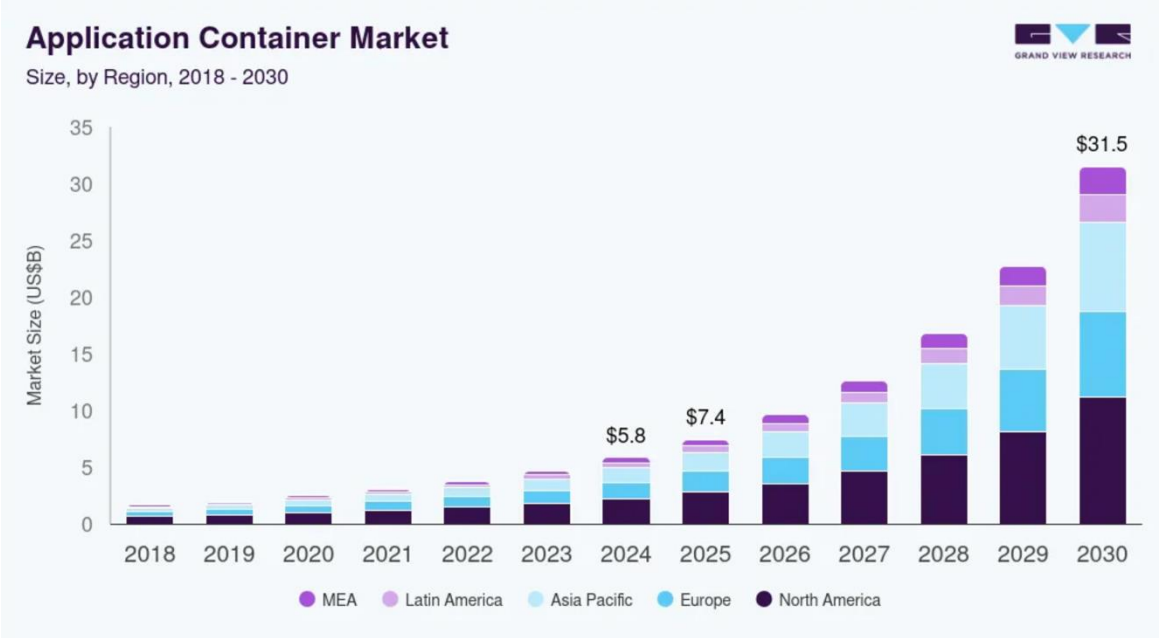
What has virtualisation to do with Distributed systems?

A ***distributed system*** is a collection of computers (nodes) working together as if they were one

Virtualisation is about dividing a single physical machine into many virtual ones. It creates ***the illusion*** that one piece of hardware is several independent computers, each running its own operating system

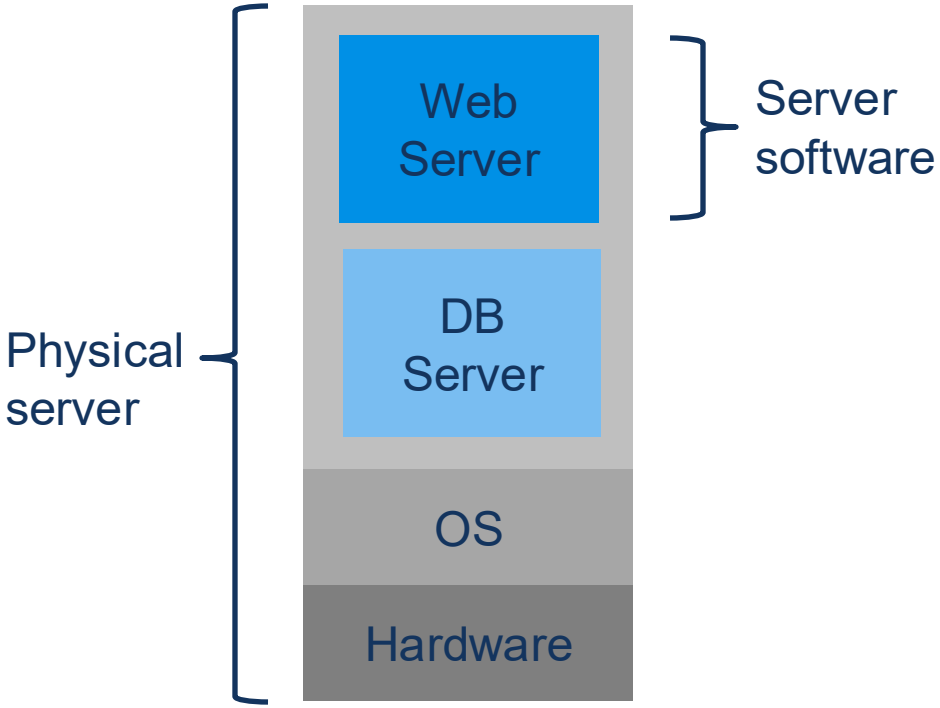
Virtualisation is the ***building block*** that enables distributed systems to scale

Data centres and cloud providers ***run vast distributed systems***, but the nodes in those systems are often virtual machines or containers



https://www.grandviewresearch.com/industry-analysis/application-container-market

Pre-virtualisation Era (1990s-early 2000s)





There was a time when each service had it owns physical server

Pre-virtualisation Era (1990s-early 2000s)
Challenges

Resource Waste

- **Severe under-utilisation**: Most servers ran at 10-20% CPU capacity
- **Peak provisioning**: Had to buy hardware for maximum expected load, which rarely occurred
- **Idle resources**: Nights, weekends, and off-peak times meant expensive hardware sitting idle
- **Memory waste**: Each server needed full OS and overhead, even for simple services

Pre-virtualisation Era (1990s-early 2000s)
Challenges

Scaling Limitations

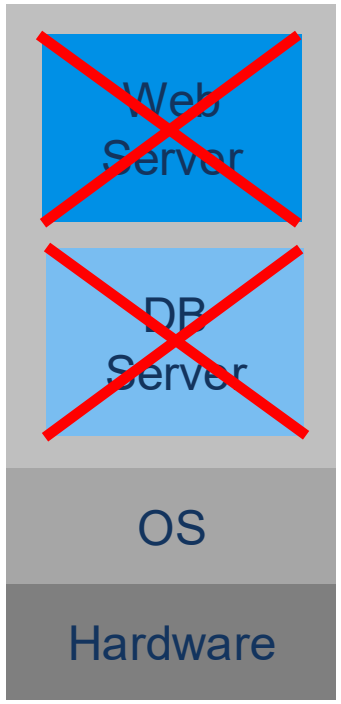
- **Hardware expenses**: Buying separate servers for each service (tens of thousands per server)
- **Data center costs**: Rack space, power, cooling, network equipment for each server
- **Energy bills**: Running and cooling many underutilized servers 24/7
- **Geographic limitations**: Replicating services meant duplicating entire physical infrastructure
- **Seasonal waste**: E-commerce sites had servers for Black Friday that sat idle 360 days/year

Pre-virtualisation Era (1990s-early 2000s)
Challenges

Operational Nightmares

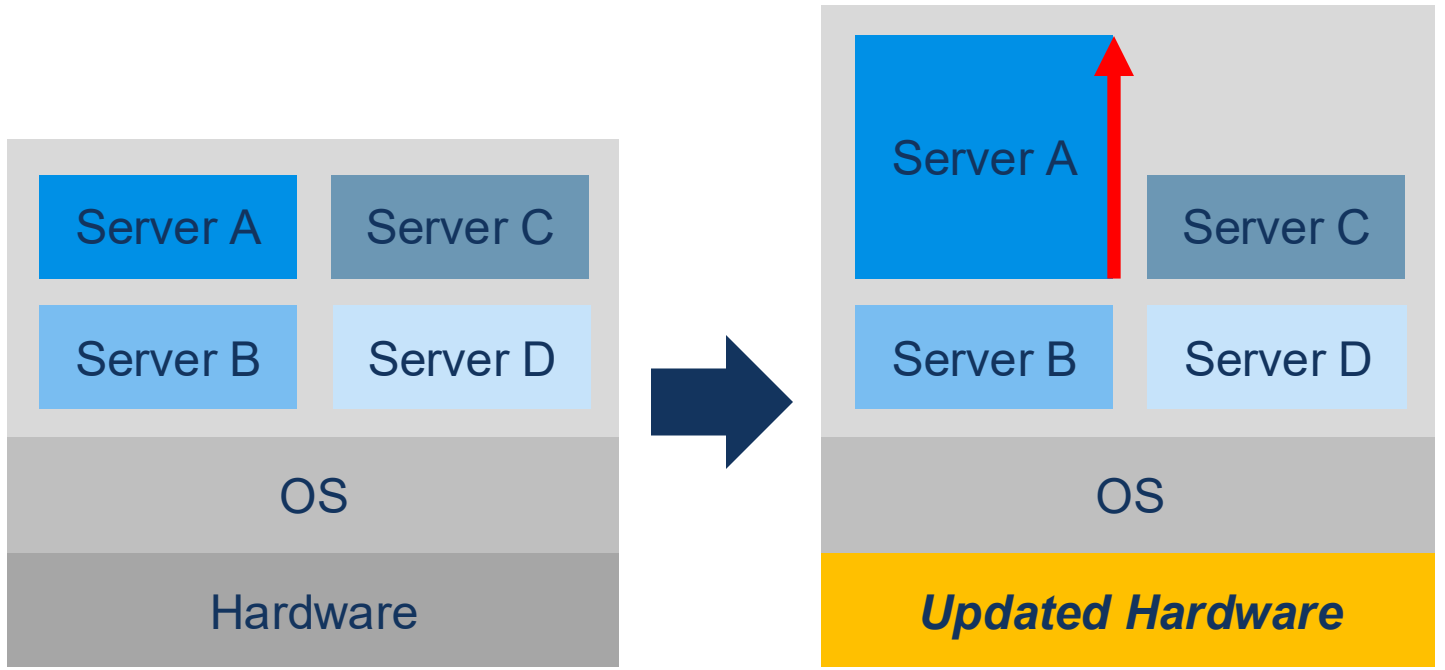
- **Slow provisioning**: Getting a new service online took weeks/months (procurement, shipping, racking, OS installation)
- **Manual everything**: Physical trips to data center for hardware issues
- **Maintenance multiplication**: OS patches, security updates, driver updates on every single server
- **Hardware failures**: When a server died, the entire service went down

Pre-virtualisation Era (1990s-early 2000s)



If the Web server crashes, the DB server might also crash

Pre-virtualisation Era (1990s-early 2000s)



Server B need more memory, we need to buy an entire new physical server
We also need to re-install all new servers SW on the new server

Rise of Virtualisation (Mid-2000s - Early 2010s)

A Virtual Machine (VM) is a software-based emulation of a physical computer that runs on another computer system, ***a computer within a computer***

It's a fully functional computing environment that exists entirely in software but behaves like a separate physical machine

A virtual machine consists of:

Virtual hardware: Simulated CPU, memory, storage, network interfaces, and other devices

Guest operating system: The OS that runs inside the VM (Windows, Linux, etc.)

Applications: Software that runs on the guest OS, just like on a physical computer

Virtual machines are created and managed by a piece of software called a **hypervisor**. The hypervisor allocates physical resources from the host computer to each VM and manages the execution of multiple VMs simultaneously

Rise of Virtualisation (Mid-2000s - Early 2010s)
Opportunities

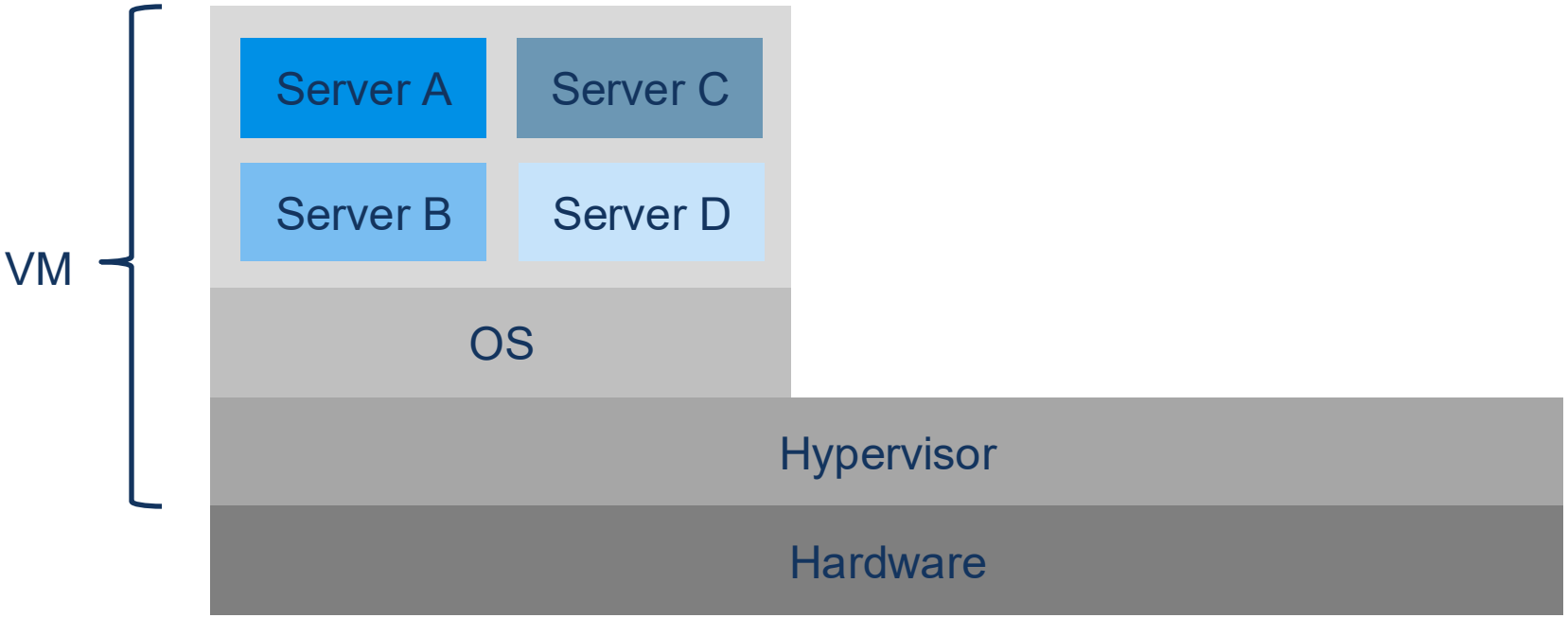
Major Improvements

- **Resource Utilization:** Improved server utilization
- **Rapid Provisioning:** New "servers" in minutes, not weeks
- **Hardware Consolidation**: 10-20 physical servers → 1 physical server with many VMs
- **Live Migration:** Move running VMs between physical hosts without downtime
- **Snapshots/Cloning:** Easy backup and replication of entire servers
- **Dynamic Resources:** Adjust CPU/RAM without buying new hardware

New Capabilities

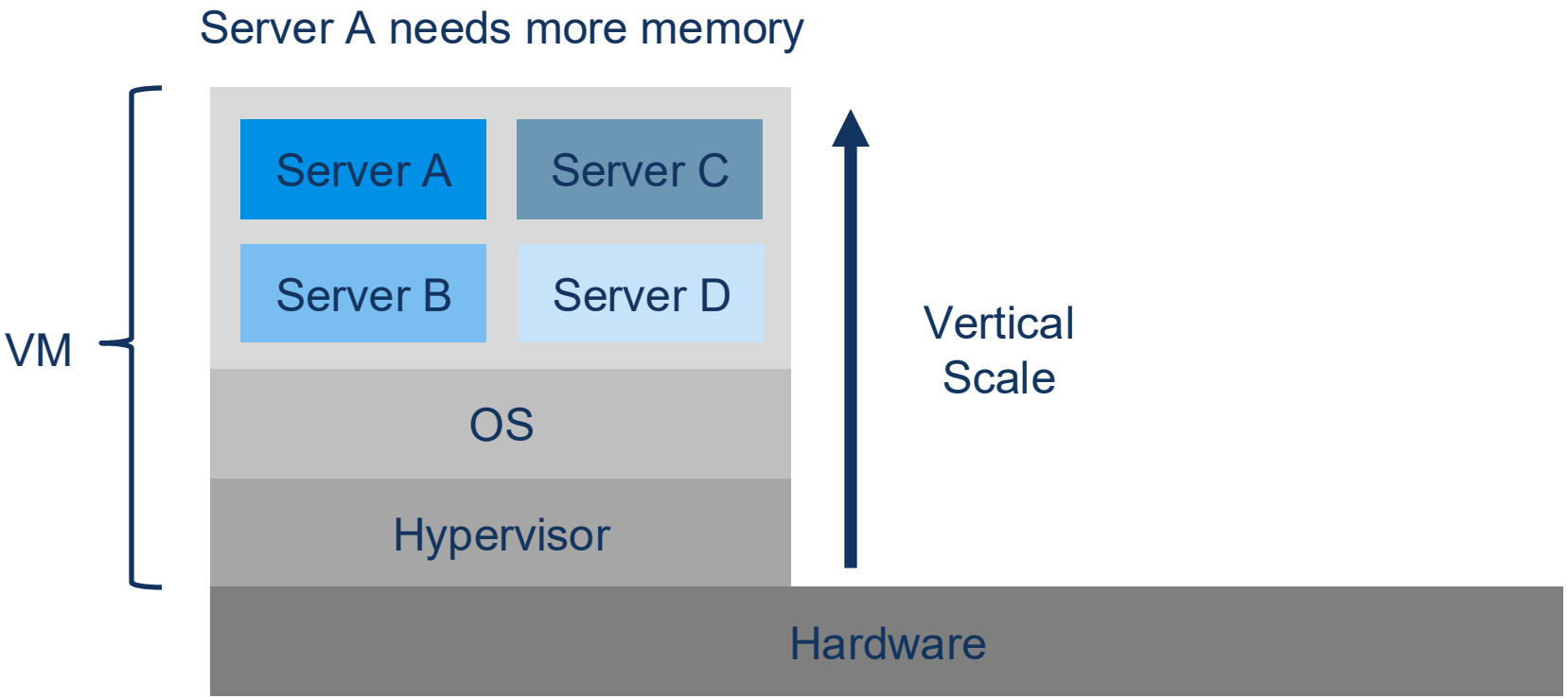
- **Disaster Recovery:** VM snapshots and replication made DR feasible
- **Server Templates:** Standardized VM images for consistency

Rise of Virtualisation (Mid-2000s - Early 2010s)



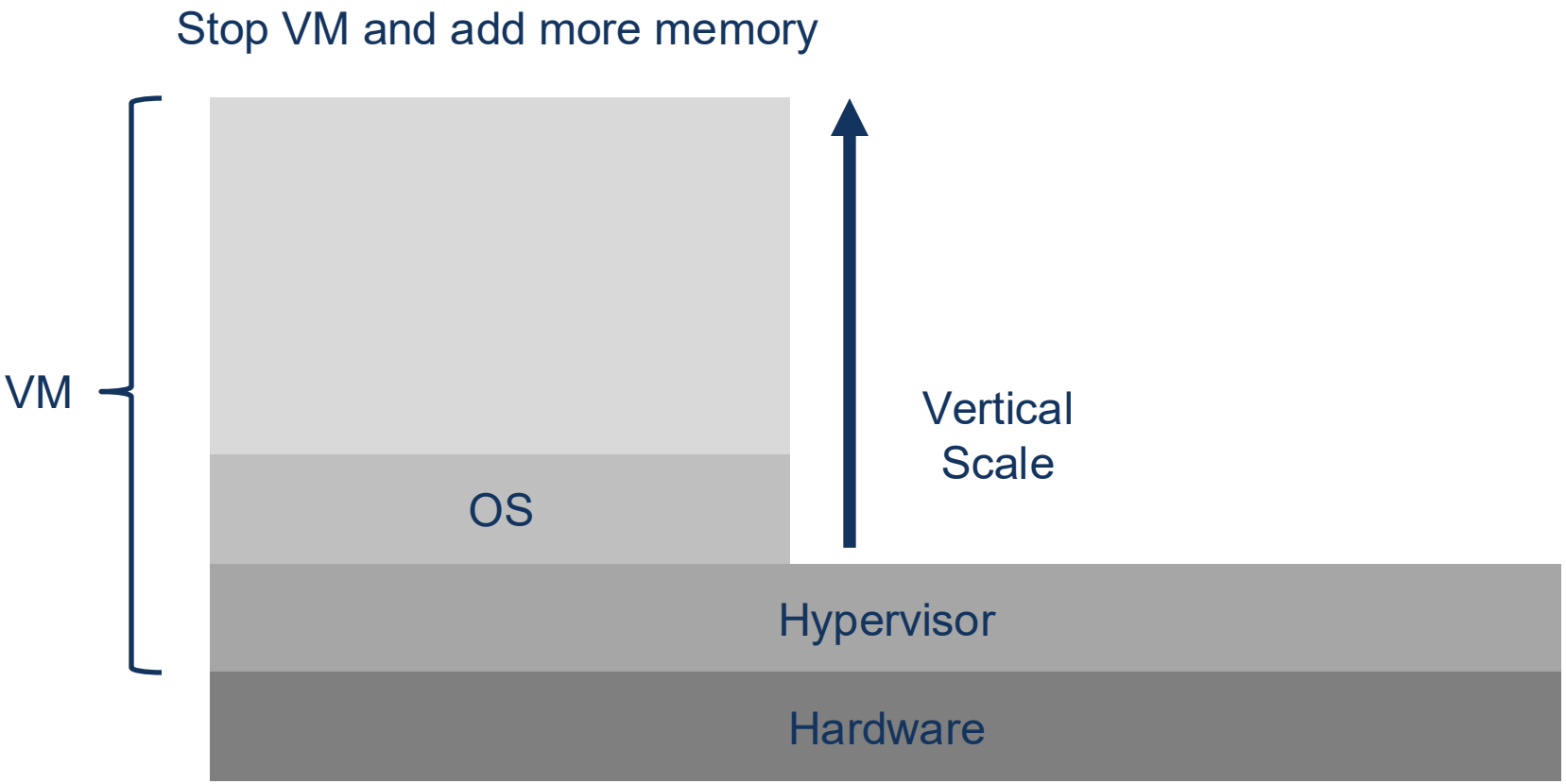
However, scaling the VM affect all running SW servers

Rise of Virtualisation (Mid-2000s - Early 2010s)



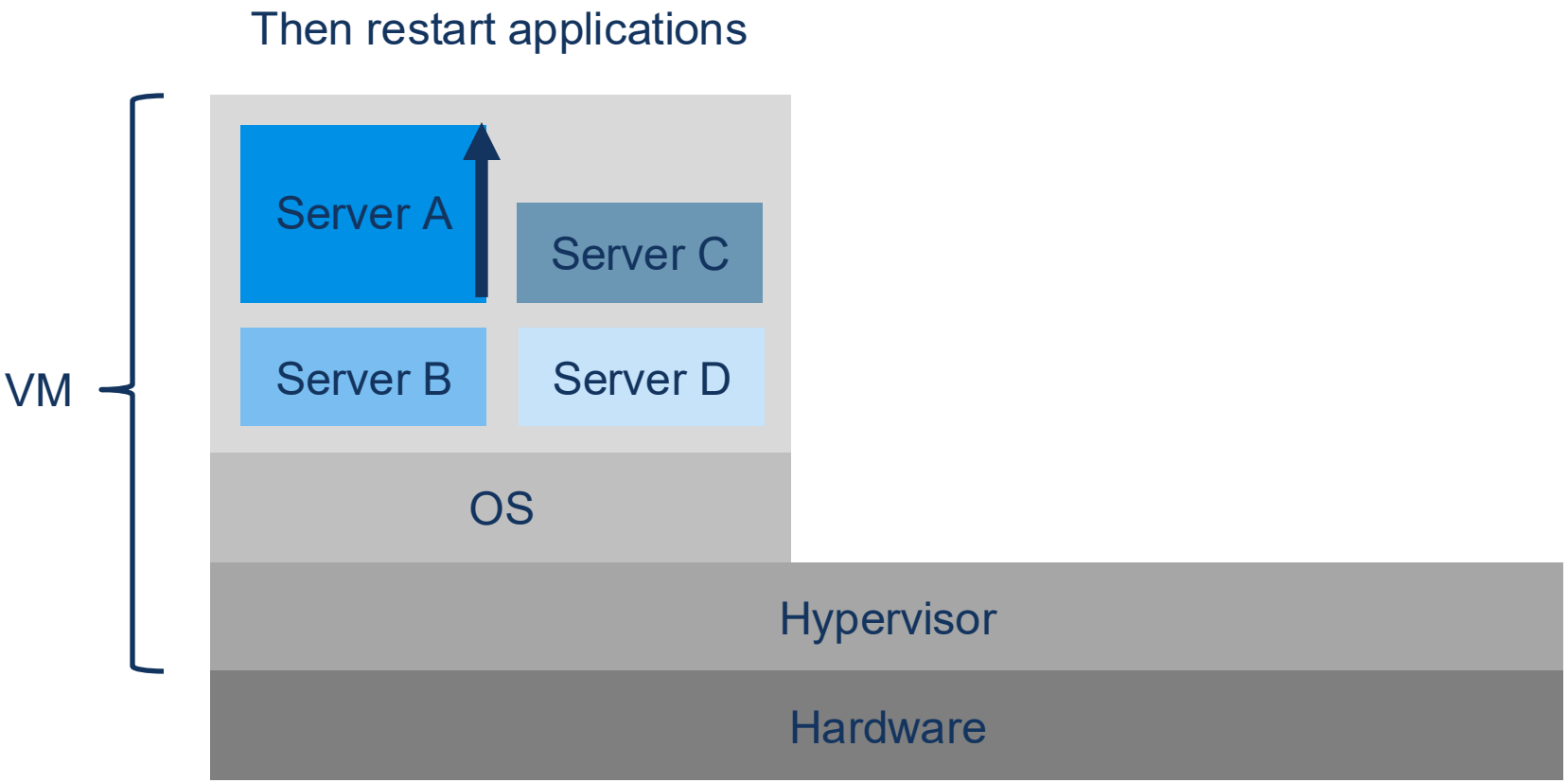
However, scaling the VM affect all running SW servers

Rise of Virtualisation (Mid-2000s - Early 2010s)



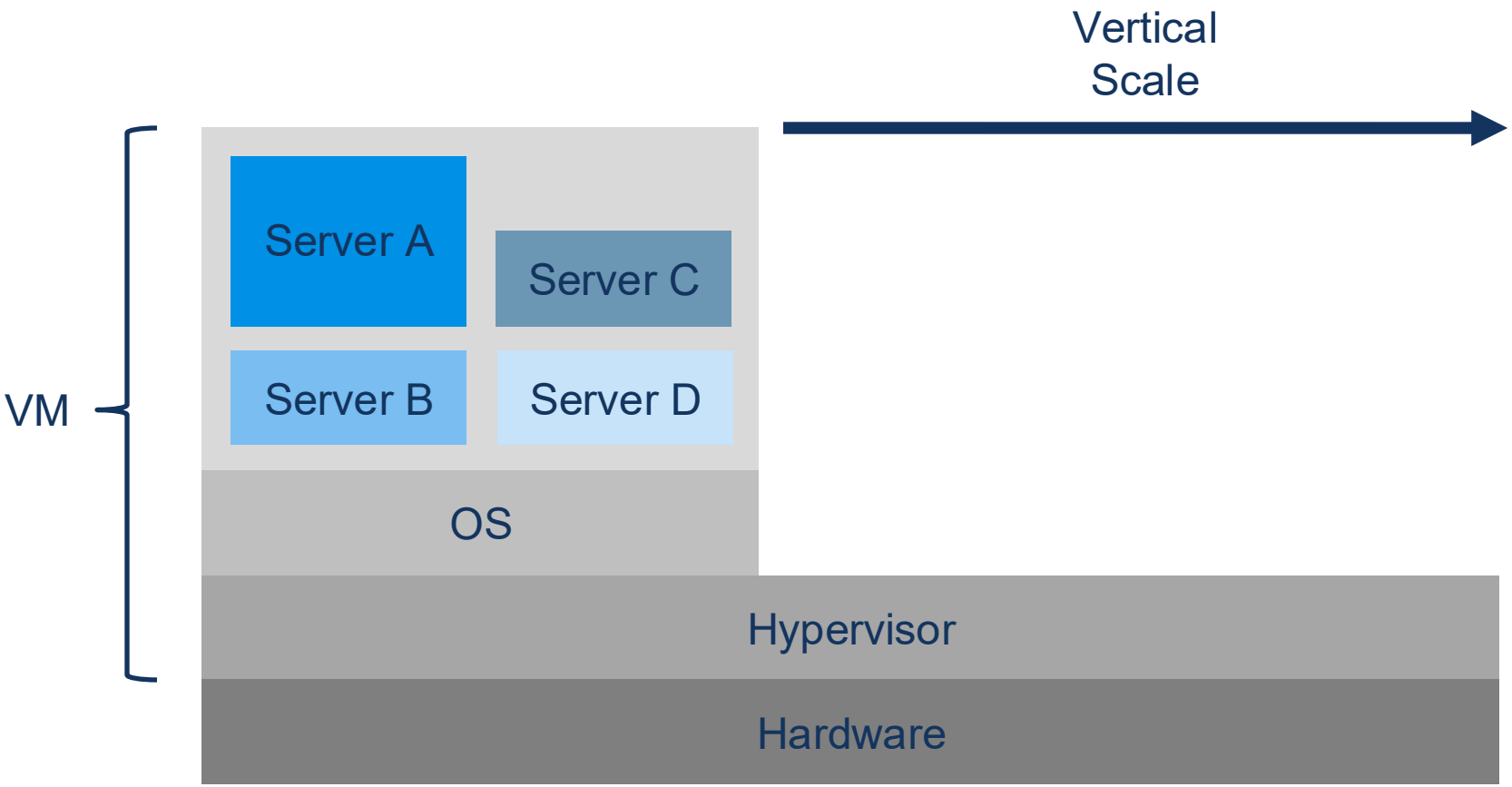
However, scaling the VM affect all running SW servers

Rise of Virtualisation (Mid-2000s - Early 2010s)

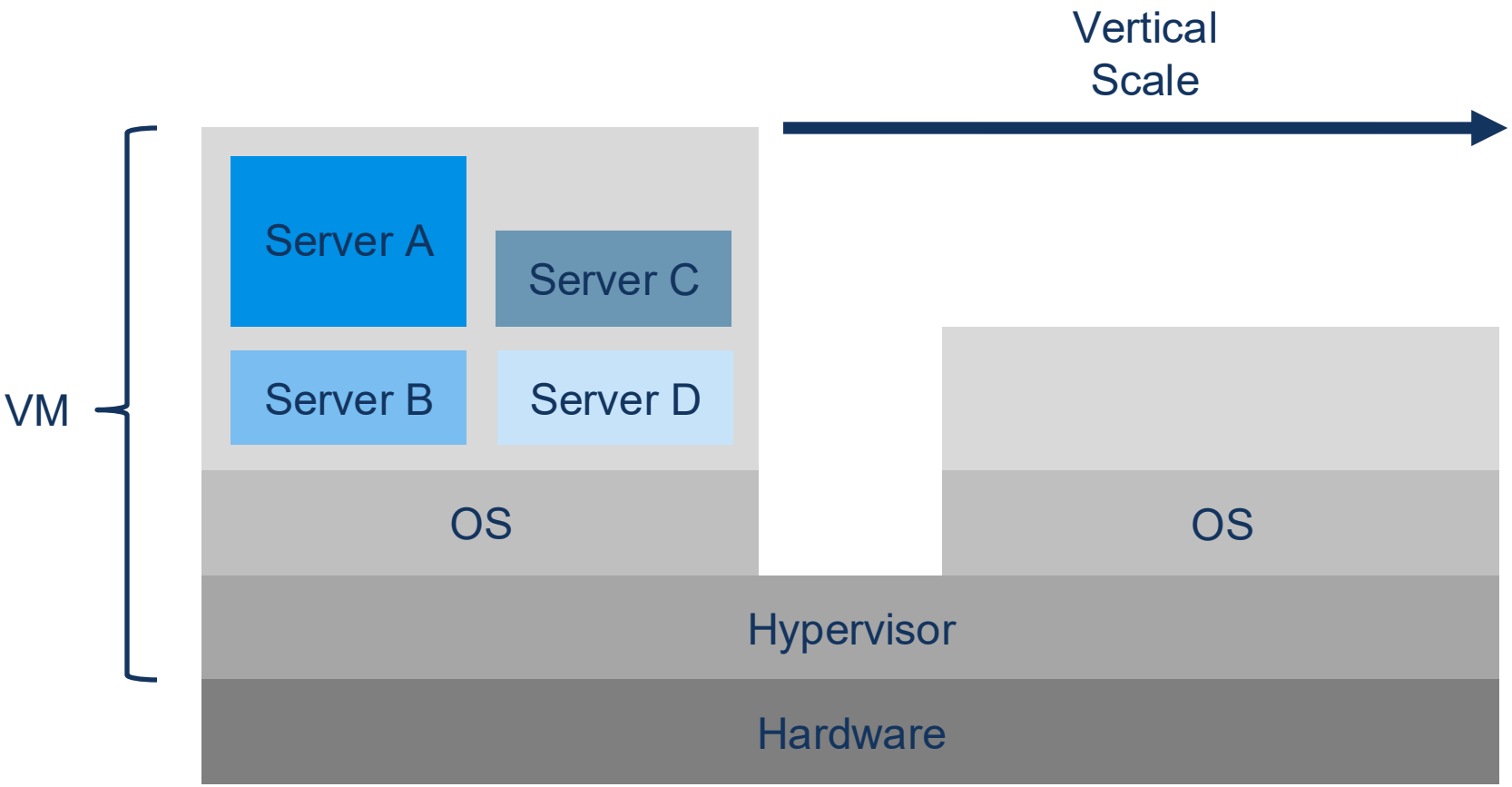


However, scaling the VM affect all running SW servers

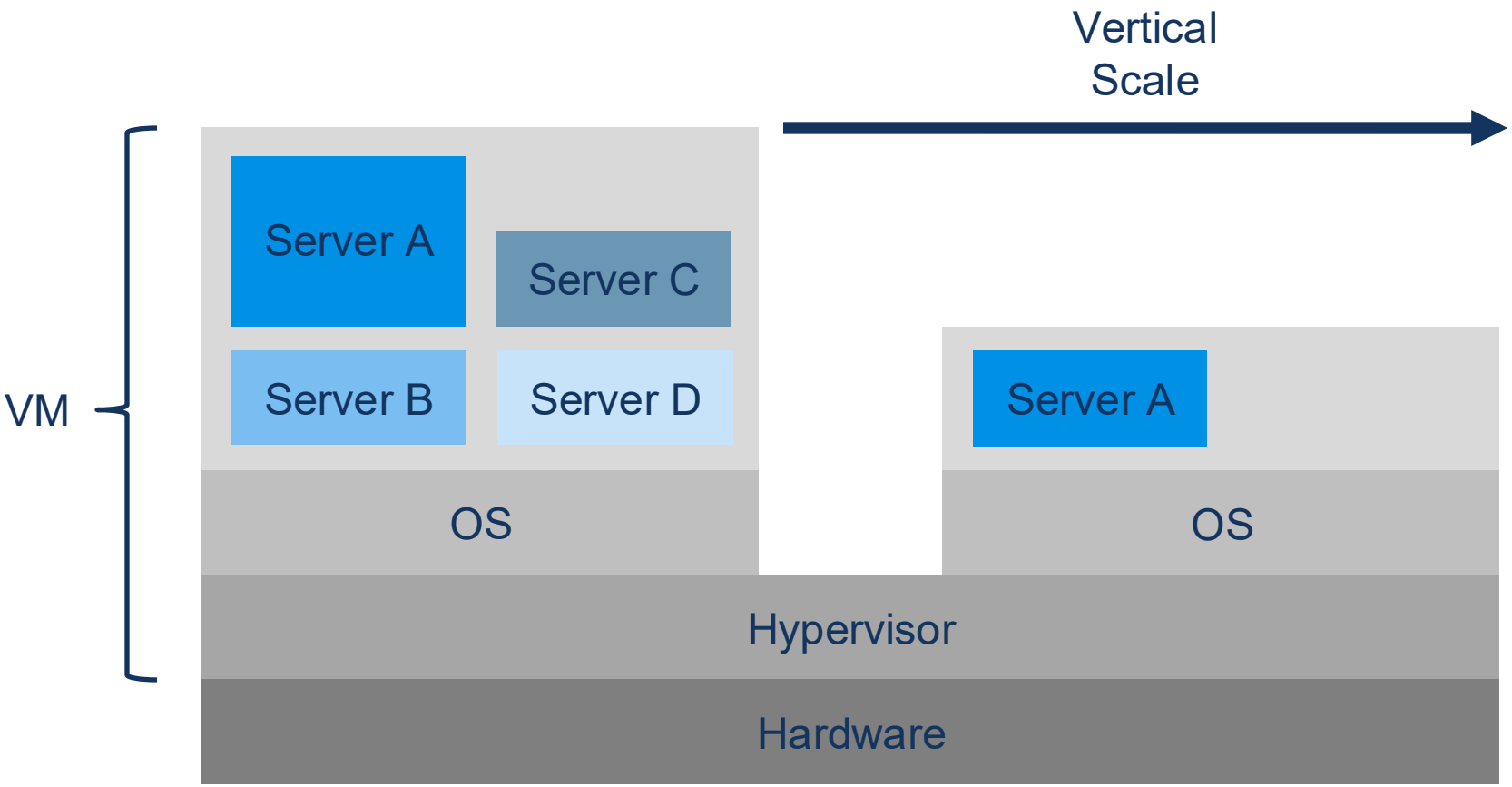
Rise of Virtualisation (Mid-2000s - Early 2010s)



Rise of Virtualisation (Mid-2000s - Early 2010s)



Rise of Virtualisation (Mid-2000s - Early 2010s)



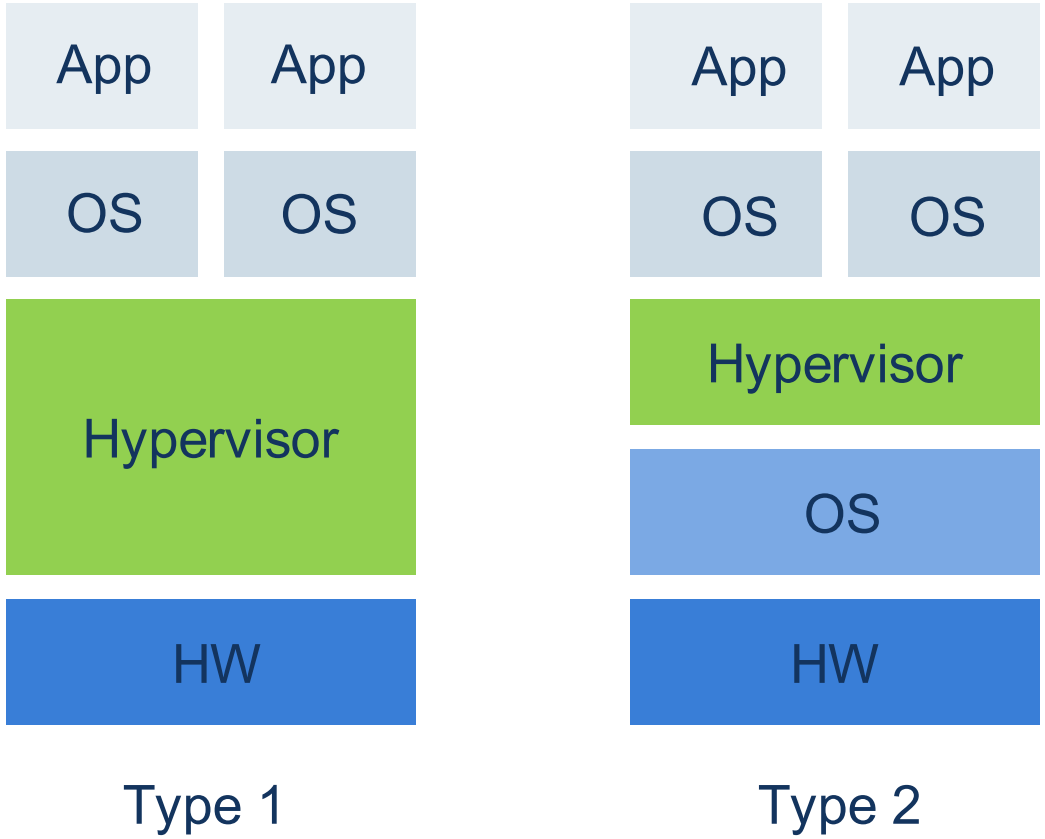
Rise of Virtualisation (Mid-2000s - Early 2010s)

Type 1 (Bare-Metal) Hypervisors

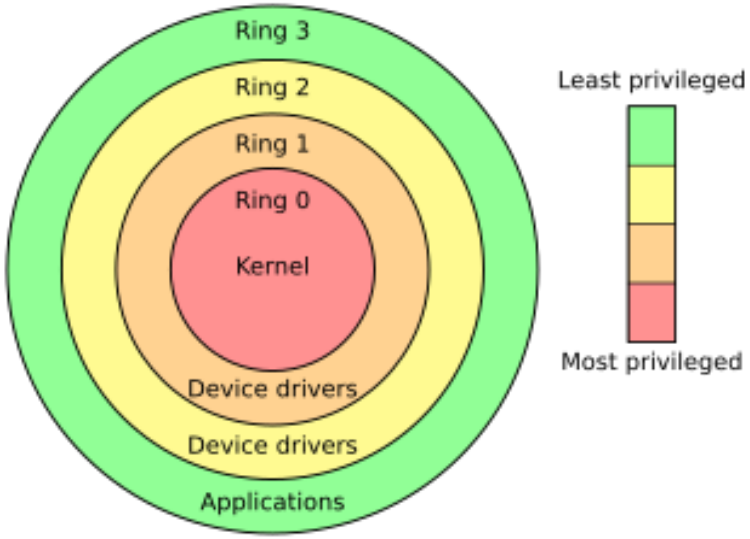
- Runs directly on physical hardware
- Direct hardware access = better performance
- Smaller attack surface = more secure
- VMware ESXi, Xen, Microsoft Hyper-V, KVM
- Used in data centers and cloud providers

Type 2 (Hosted) Hypervisors

- Runs on top of a host operating system
- Easier to install (just an application)
- Performance overhead from host OS
- VMware Workstation, VirtualBox, Parallels
- Desktop virtualization, good for development/testing



How does a Hypervisor work?



CPU Virtualisation

- Modern processors include hardware virtualisation extensions (Intel VT-x, AMD-V) that the hypervisor leverages. When a VM executes instructions, the hypervisor uses these extensions to:
- Trap privileged operations that require direct hardware access
- Execute most VM instructions directly on the physical CPU for performance
- Switch CPU context between different VMs rapidly (CPU scheduling)

Privilege Levels

- VMs run at a lower privilege level than the hypervisor itself.
- Any attempt by a VM to execute privileged instructions (like directly accessing hardware) triggers a trap to the hypervisor, which then decides how to handle the request.

How does a Hypervisor work?

Memory Virtualisation

- Each VM believes it has access to a continuous block of physical memory starting at address 0
- The hypervisor maps these "guest physical addresses" to actual physical memory locations
- It uses techniques like shadow page tables or Extended Page Tables (EPT) to manage these mappings

Memory Isolation

- Each VM can only access memory pages allocated to it by the hypervisor
- The memory management unit (MMU) and hypervisor work together to prevent VMs from reading or writing each other's memory

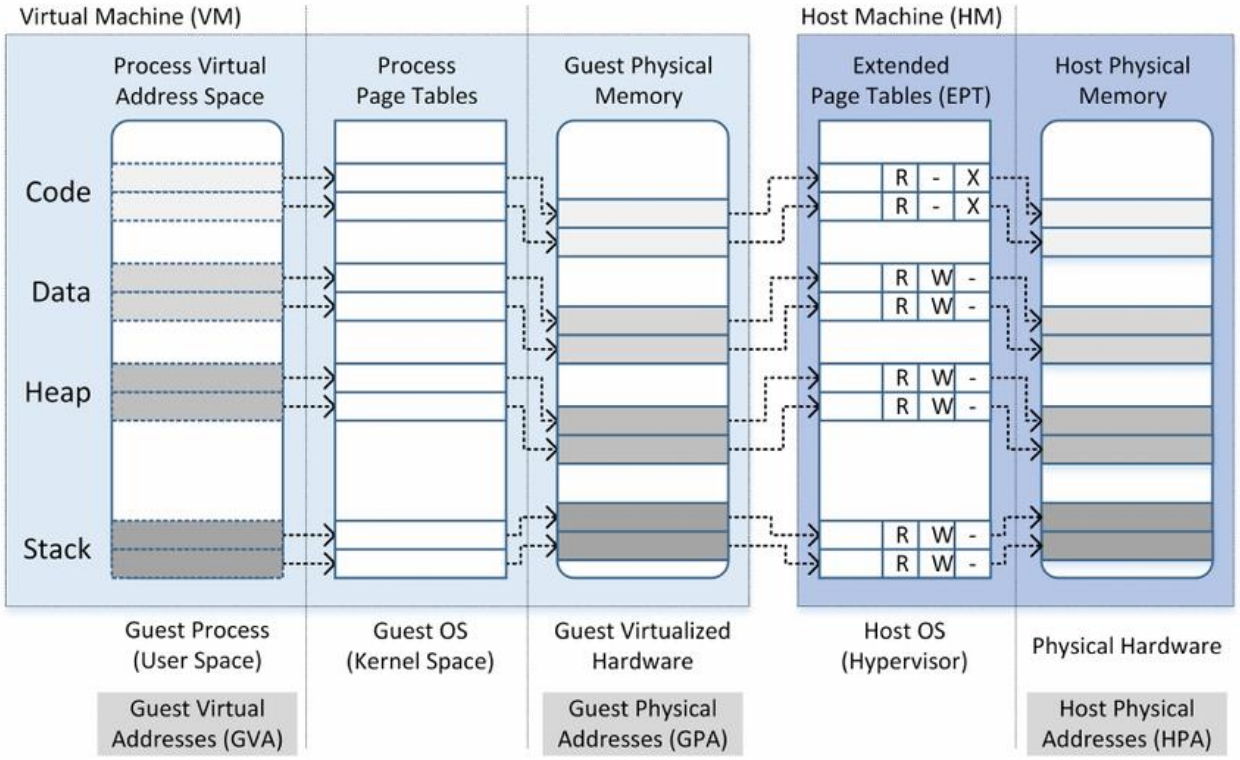


Figure source: Luțaș, Andrei & Coleșa, Adrian & Lukacs, Sandor & Lutas, Dan. (2015). U-HIPE: hypervisor-based protection of user-mode processes in Windows. Journal of Computer Virology and Hacking Techniques. 10.1007/s11416-015-0237-z.

Full-virtualisation

A virtualisation technique (CPU traps or hardware extensions) where the guest operating system runs unmodified, believing it has complete control of the hardware

The hypervisor emulates the underlying hardware and intercepts privileged instructions, translating them into safe operations on the host system

KVM (with QEMU) and VMware ESXi use hardware-assisted virtualisation (Intel VT-x / AMD-V) to run multiple unmodified guest OSes efficiently

Trade-offs

- No OS modification required — any operating system can run as a guest
- Hardware emulation and trapping ensure isolation and compatibility
- Higher overhead compared to para-virtualisation, due to instruction emulation and device abstraction.

Para-virtualisation

A virtualisation technique where the guest operating system (OS) is modified to communicate directly with the hypervisor, instead of fully emulating hardware

The guest OS uses hypercalls (instead of privileged CPU instructions) to interact with the hypervisor for privileged operations such as CPU scheduling and memory management

For example, Xen project, para-virtualised guests *(DomUs*) run a modified Linux kernel that communicates directly with the Xen hypervisor, while I/O operations are handled through *Dom0* using front-end and back-end drivers

Trade-offs

- Higher performance — less emulation overhead than full virtualisation
- However, requires guest OS kernel modification and maintenance.

Hardware-Assisted Virtualisation

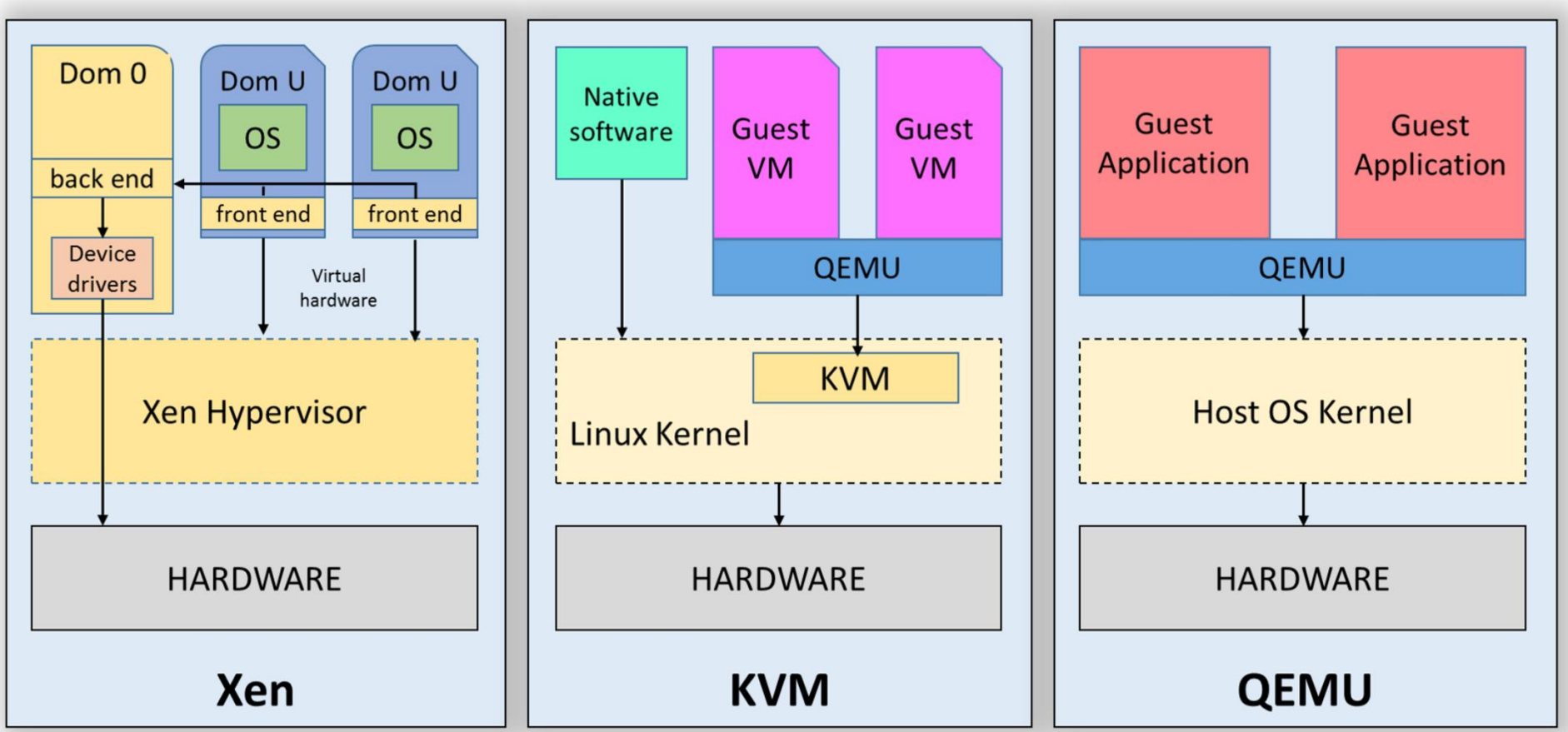
A virtualisation technique that uses CPU hardware extensions (such as **Intel VT-x** or **AMD-V**) to accelerate virtualisation and reduce hypervisor overhead

- The CPU itself traps and manages privileged instructions from guest operating systems, allowing them to run unmodified while maintaining strong isolation
- Guest OS runs at near-native speed without kernel modification
- Supports full virtualisation of CPU and memory, often combined with para-virtualised I/O drivers for efficiency

Examples

- Xen (HVM mode)
- KVM (Kernel-based Virtual Machine)
- Microsoft Hyper-V, VMware ESXi

Xen vs KVM vs QEMU

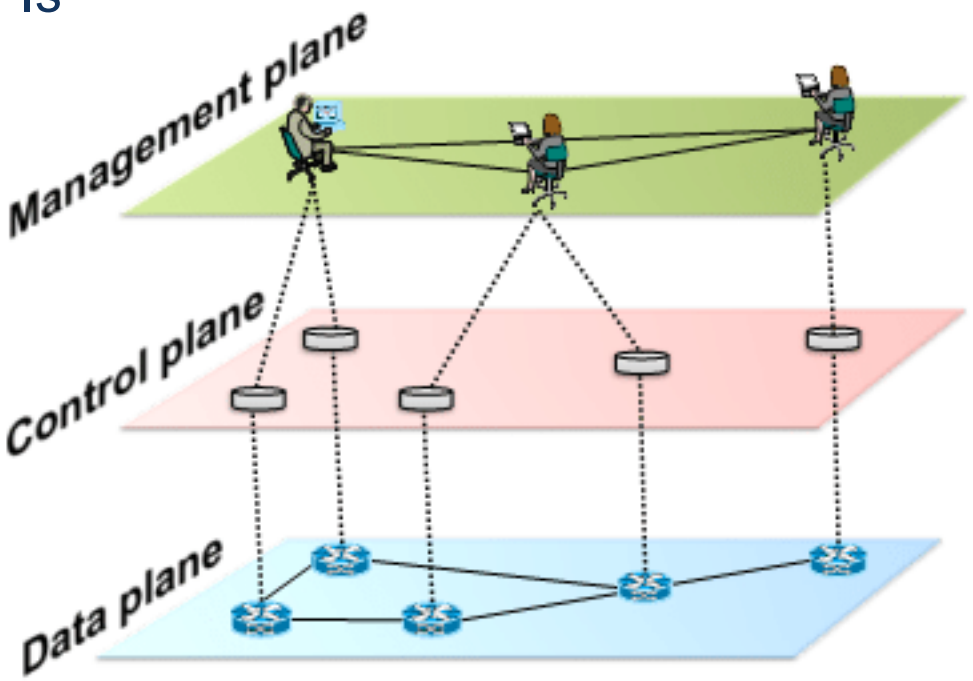


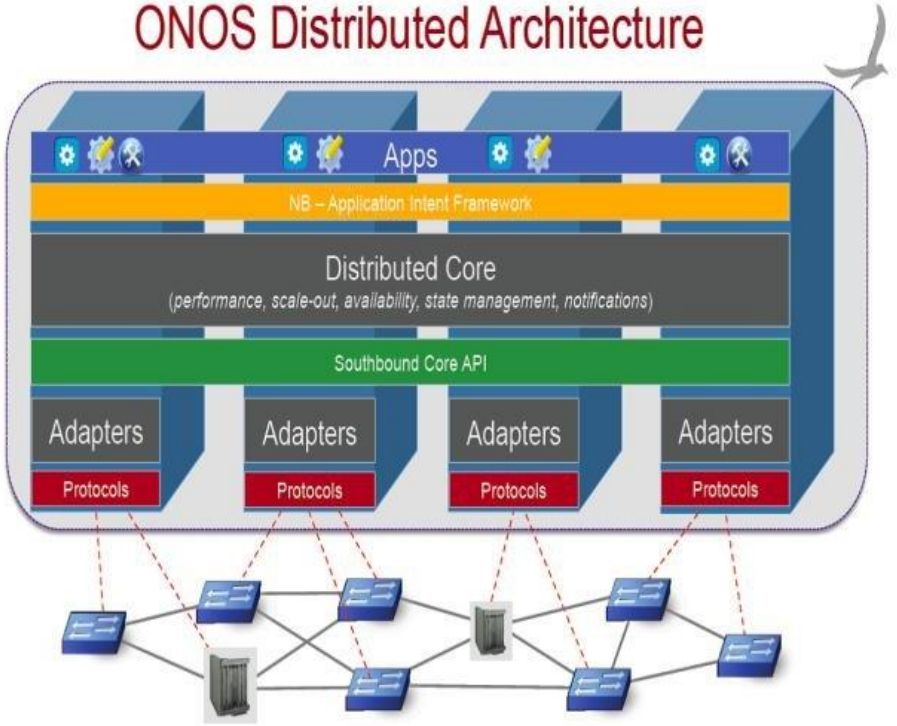
Source: Song, Yang & Wang, Haoliang & Soyata, Tolga. (2015). *Hardware and Software Aspects of VM-Based Mobile-Cloud Offloading*. 10.4018/978-1-4666-8662-5.ch008.

Network Virtualisation

Abstracting physical networking into software-based logical networks. Enables networks to be created, managed, and scaled independently of hardware

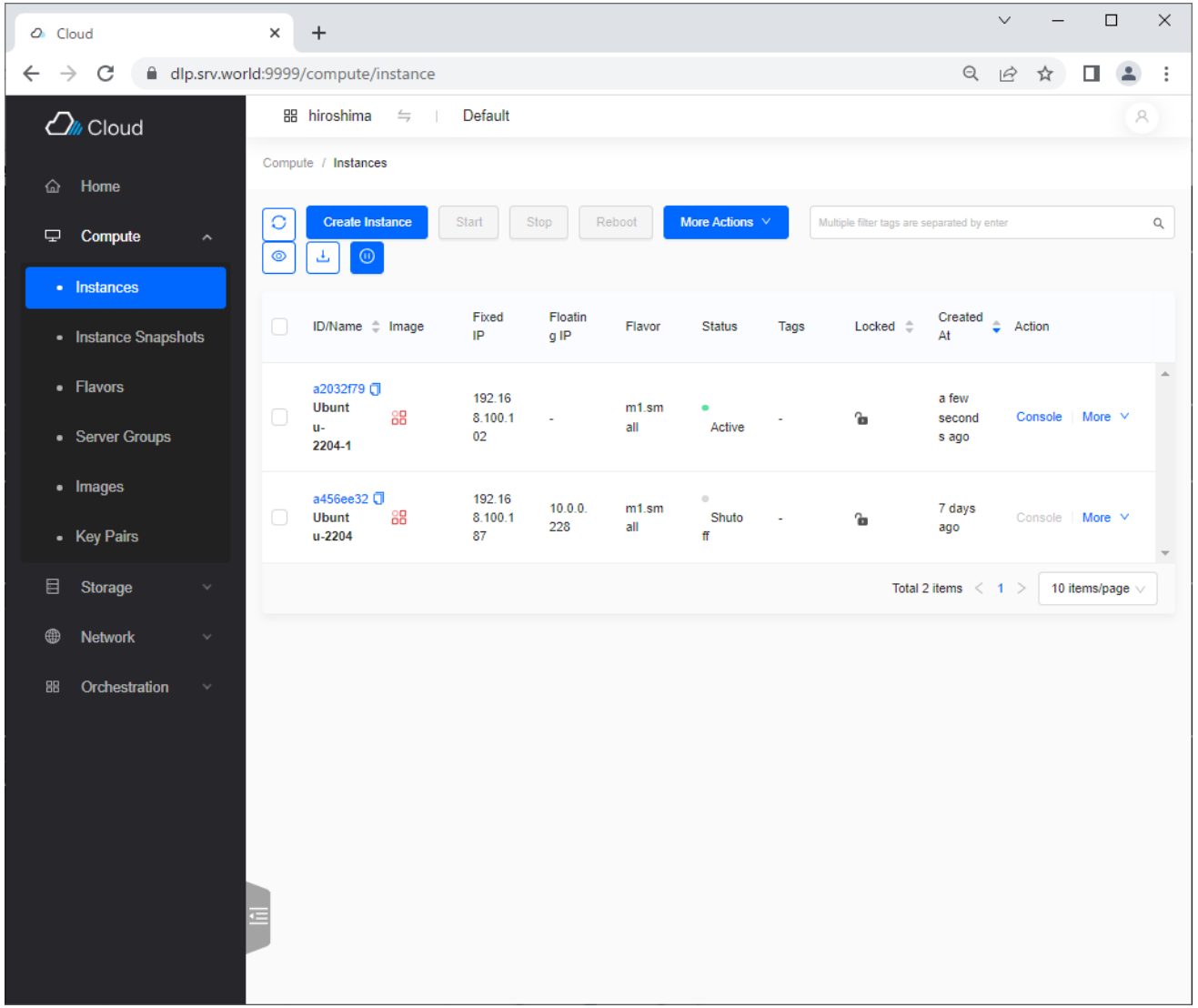
- Makes networking as dynamic and programmable (Software-Defined Network)
- Virtual switches, routers, firewalls; separation of control plane and data plane; programmable via APIs



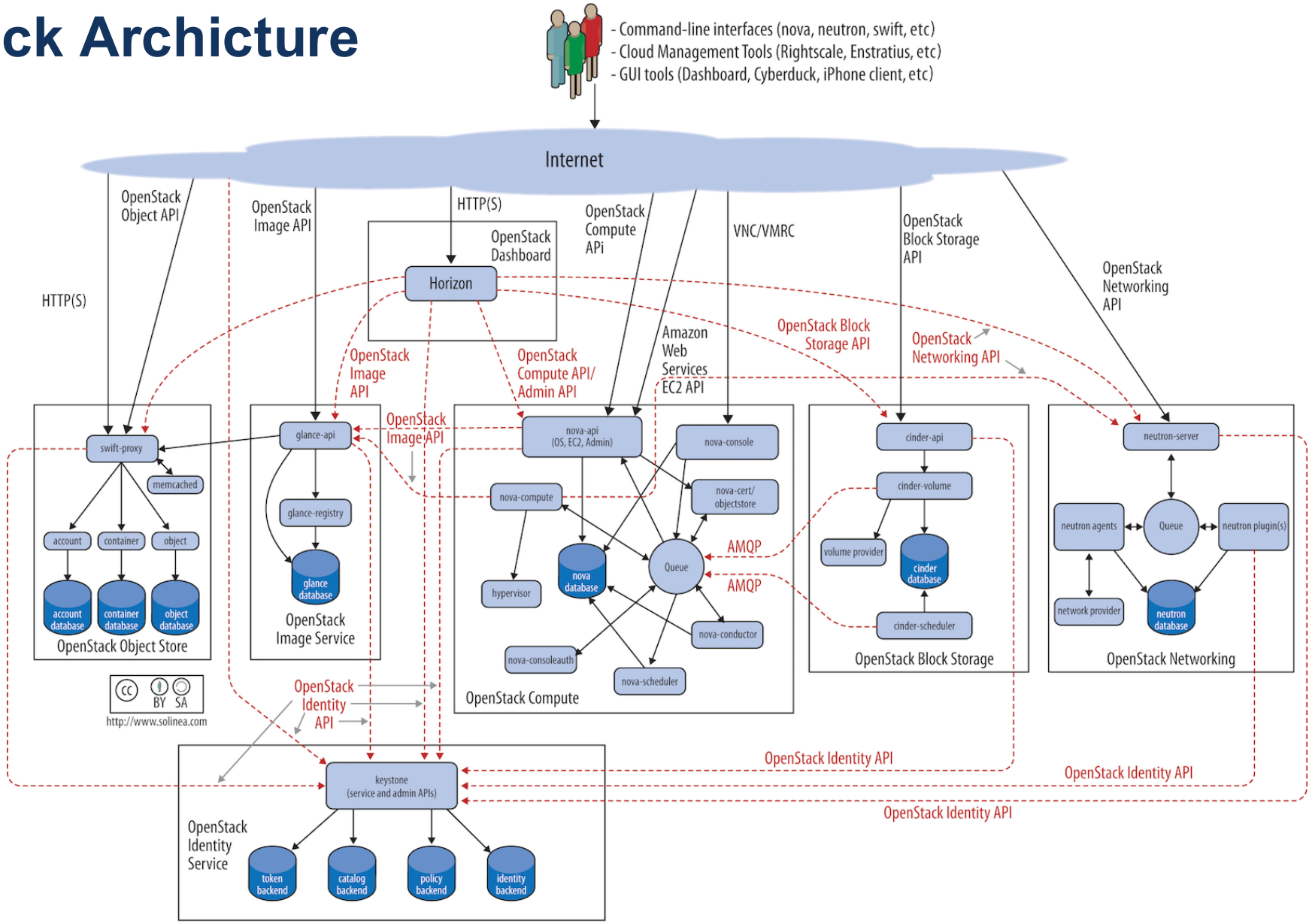


OpenStack

- Open source cloud computing platform for building public and private clouds
- Provides Infrastructure-as-a-Service (IaaS) compute, storage, networking on demand
- Widely adopted by telecoms, research institutions, and enterprises for private cloud deployments



OpenStack Archicture



Creating an VM instance

User + Keystone (Authentication)
1. Dashboard/CLI sends credentials to Keystone.
2. Keystone validates credentials, returns auth-token.
3. Dashboard/CLI sends “launch instance” to nova-api.
4. Nova-api checks auth-token with Keystone.
5. Keystone validates token, returns roles/permissions.

Nova API + Scheduler + Database
6. Nova-api writes request to nova-database.
7. Nova-database creates initial instance entry.
8. Nova-api places request to scheduler via queue.
9. Nova-scheduler reads request from queue.
10. Nova-scheduler queries nova-database for hosts.
11. Nova-scheduler filters/weighs and selects host.
12. Nova-scheduler updates DB with host ID.
13. Nova-scheduler sends cast request to nova-compute.

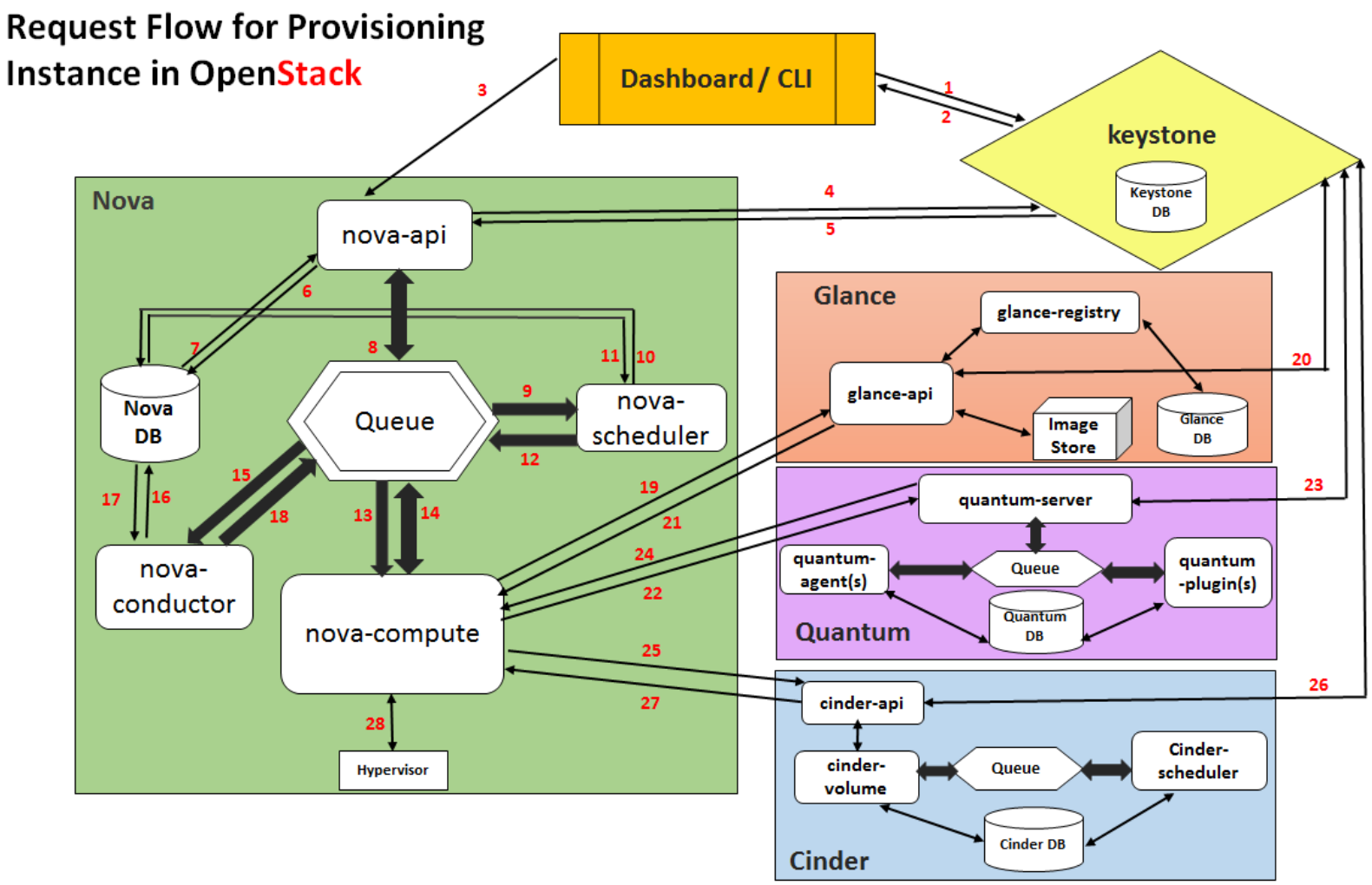
Nova Compute + Conductor
14. Nova-compute picks request from queue.
15. Nova-compute asks nova-conductor for instance info.
16. Nova-conductor queries nova-database.
17. Nova-database returns instance information.
18. Nova-conductor sends info to nova-compute.

Glance (Image Service)
19. Nova-compute requests image from glance-api.
20. Glance-api validates token with Keystone.
21. Nova-compute receives image metadata.

Neutron (Networking)
22. Nova-compute requests network from neutron-api.
23. Neutron validates token with Keystone.
24. Nova-compute receives network configuration.

Cinder (Block Storage)
25. Nova-compute requests volume from cinder-api.
26. Cinder-api validates token with Keystone.
27. Nova-compute receives volume information.

Hypervisor (Final Launch)
28. Nova-compute instructs hypervisor to launch VM.

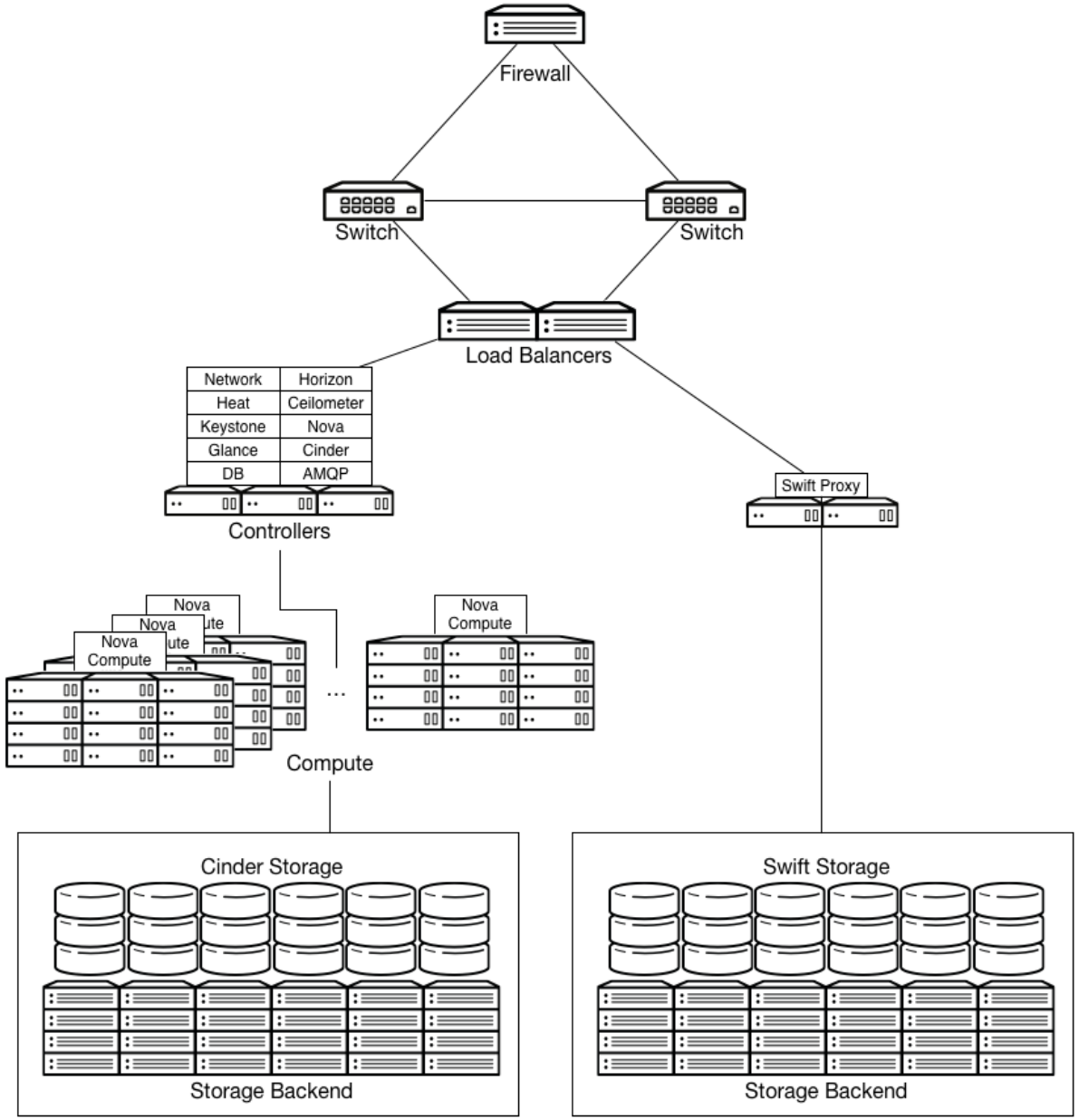


Source: https://maestropandy.wordpress.com/2016/05/26/openstack-instance-creation-request-work-flow

OpenStack Deployment

Core design principles

- Keep control-plane services stateless and centralise shared state in external DB
  - No Paxos/Raft is needed
  - Easy horizontal scaling and rolling upgrades
- Coordination/RPC flows through an HA message queue (RabbitMQ)
- Control-plane services can be scaled horizontally, run as many as you want behind a load balancer

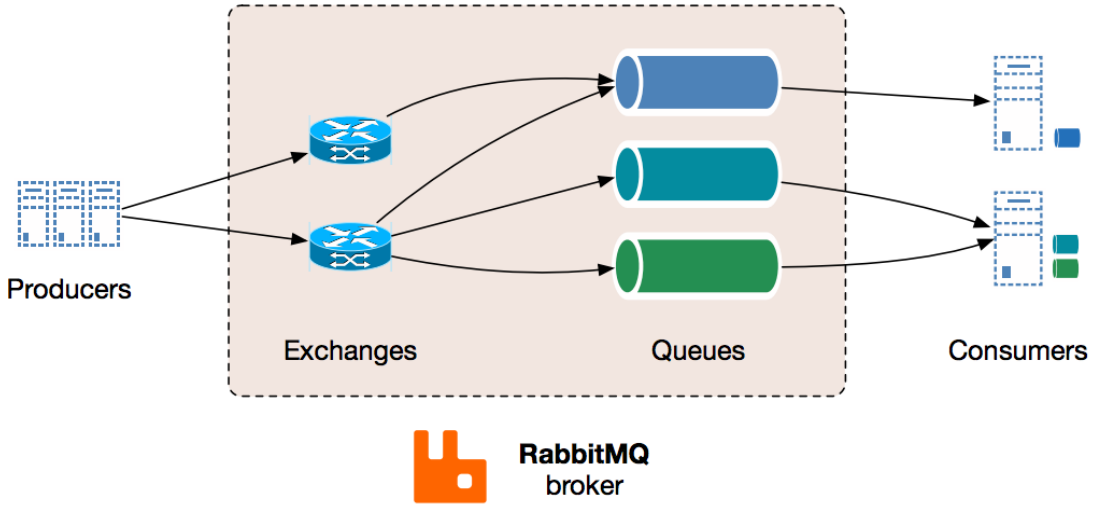


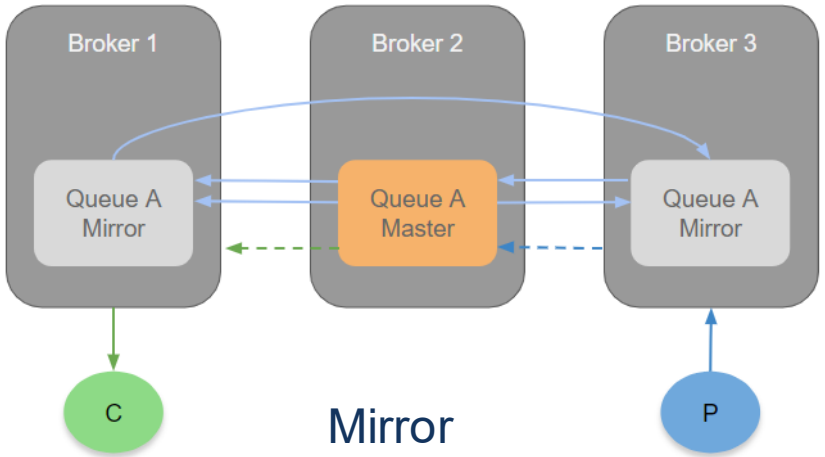
Source: https://docs.openstack.org/arch-design/use-cases/use-case-general-compute.html

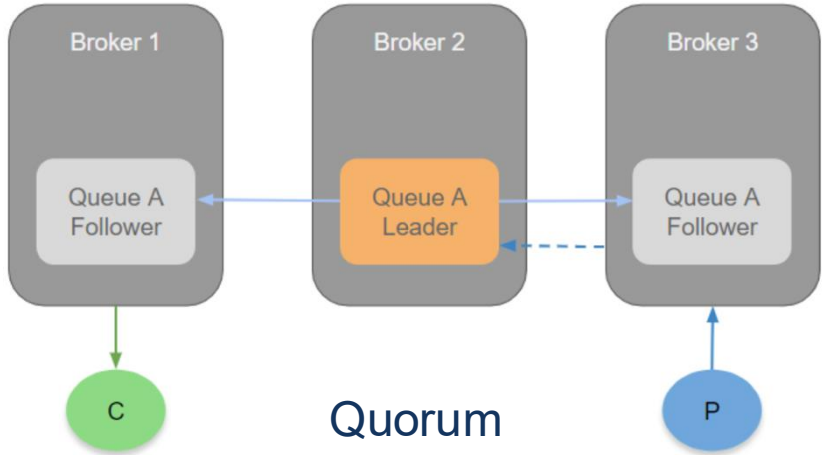
RabbitMQ

RabbitMQ offers two types of replicated queue: mirrored queues (HA queues) and quorum queues

- Quorum queues are CP and tolerate the loss of brokers as long as a quorum (majority) remains functioning.
- Queues can be mirrored and configured for either availability (AP) or consistency (CP)







Source: https://www.rabbitmq.com/blog/2020/06/23/quorum-queues-local-delivery

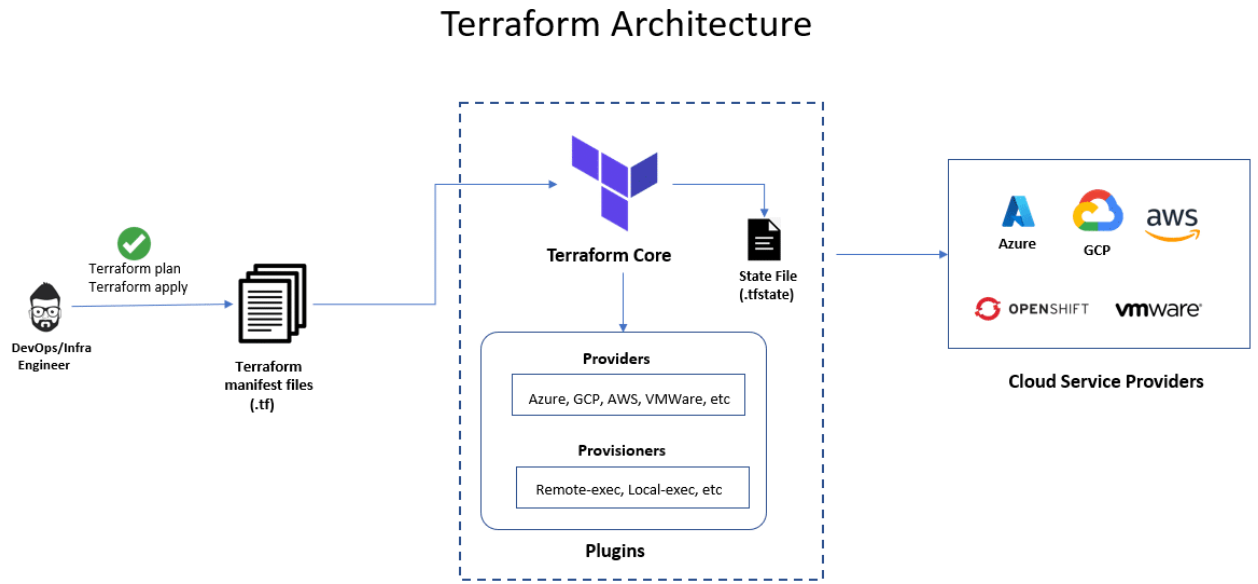
Deploying applications on OpenStack

1. **Provision a VM**
   Example: 2 vCPUs, 2 GiB RAM, 20 GiB disk
2. **Choose Ubuntu 22.04 image**
3. **Access the VM**
   SSH into the instance using key pair
4. **Install dependencies**
   Update system packages
   Install Apache/Nginx, MySQL/MariaDB, PHP (LAMP/LEMP stack)
5. **Deploy WordPress**
   Download WordPress package
   Configure database & credentials
   Place files in web root
6. **Access the application**
   Visit <VM_Floating_IP> in browser
   Complete WordPress setup wizard

Infrastructure-as-Code (IaaS)

Managing and provisioning infrastructure (servers, networks, storage, etc.) using machine-readable configuration files instead of manual setup

- Treat infrastructure like software, version-controlled, testable, repeatable
- Typically, run as a CLI that run locally or in CI/CD pipelines




```
provider "aws" {
  region = "us-west-2"
}

data "aws_ami" "ubuntu" {
  most_recent = true

  filter {
    name = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }

  owners = ["099720109477"] # Canonical
}

resource "aws_instance" "app_server" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.micro"

  tags = {
    Name = "learn-terraform"
  }
}
```

```
$ terraform apply
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 1s [id=ami-0026a04369a3093cc]

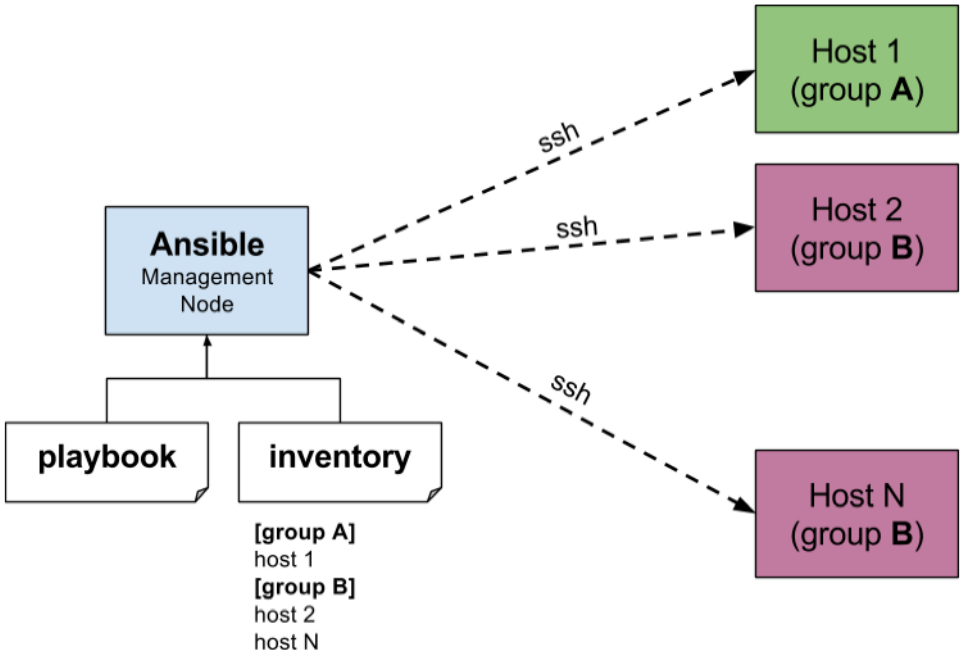
Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.app_server will be created
  + resource "aws_instance" "app_server" {
      + ami                                  = "ami-0026a04369a3093cc"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
```

Ansible

Ansible is an open-source automation platform used to configure systems, deploy applications, and manage infrastructure



Inventory

```
all:
  children:
    wordpress_servers:
      hosts:
        wp1:
          ansible_host: 192.168.1.60
          ansible_user: ubuntu
          ansible_port: 22
          ansible_ssh_private_key_file: /home/johan/.ssh/id_rsa
```

Playbook

```
- name: Install WordPress with MySQL and Apache
  hosts: wordpress_servers
  become: true

  vars:
    mysql_root_password: "StrongRootPassword123"
    wp_db_name: "wordpress"
    wp_db_user: "wp_user"
    wp_db_password: "WpPassword123!"
    wp_site_dir: "/var/www/html"

  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: yes

    - name: Install required packages
      ansible.builtin.apt:
        name:
          - apache2
          - mysql-server
          - php
          - php-mysql
          - libapache2-mod-php
          - php-cli
          - php-curl
          - php-gd
          - php-mbstring
          - php-xml
          - php-xmlrpc
          - unzip
          - wget
        state: present

    - name: Ensure MySQL service is running
      ansible.builtin.service:
        name: mysql
        state: started
        enabled: yes

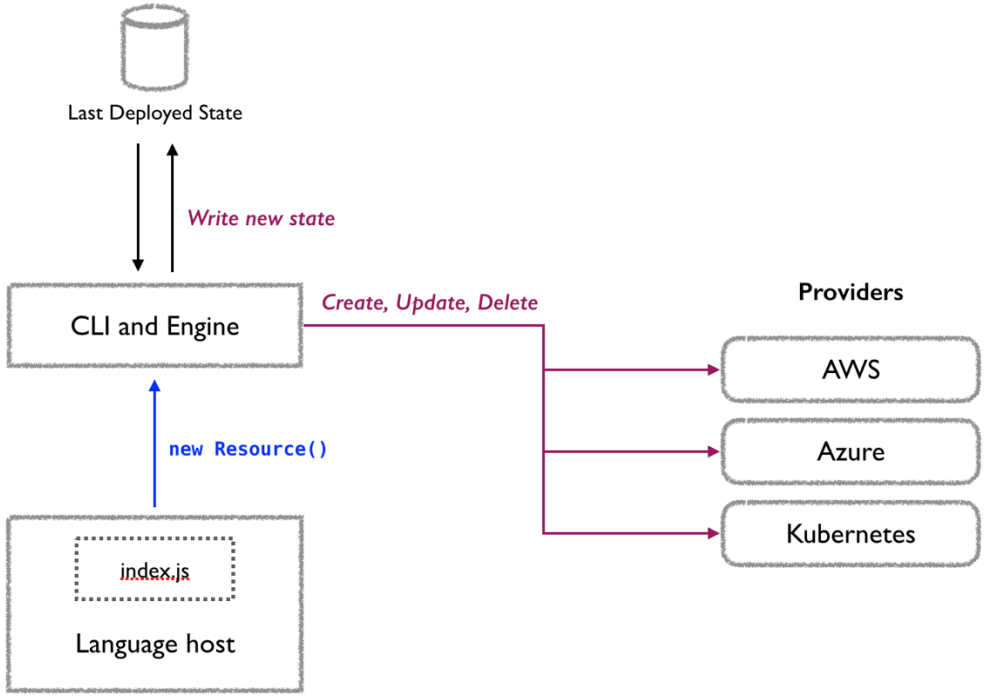
    - name: Create WordPress database
      community.mysql.mysql_db:
        name: "{{ wp_db_name }}"
        state: present
        login_user: root
        login_password: "{{ mysql_root_password }}"
      ignore_errors: yes  # First run might set root password manually

    - name: Create WordPress MySQL user
      community.mysql.mysql_user:
        name: "{{ wp_db_user }}"
        password: "{{ wp_db_password }}"
        priv: "{{ wp_db_name }}.*:ALL"
        state: present
        login_user: root
        login_password: "{{ mysql_root_password }}"
      ignore_errors: yes

.....
```

Pulumi

Unlike traditional IaC tools that require domain-specific languages, ***Pulumi*** supports general-purpose languages such as TypeScript, Python, or Go




```go
package main

import (
  "github.com/pulumi/pulumi-aws/sdk/v7/go/aws/ec2"
  "github.com/pulumi/pulumi/sdk/v3/go/pulumi"
)

func main() {
    pulumi.Run(func(ctx *pulumi.Context) error {
        group, err := ec2.NewSecurityGroup(ctx, "web-sg", &ec2.SecurityGroupArgs{
            Description: pulumi.String("Enable HTTP access"),
            Ingress: ec2.SecurityGroupIngressArray{
             ec2.SecurityGroupIngressArgs{
                Protocol:   pulumi.String("tcp"),
                FromPort:   pulumi.Int(80),
                ToPort:    pulumi.Int(80),
                CidrBlocks: pulumi.StringArray{pulumi.String("0.0.0.0/0")},
             }
          },
    })
    if err != nil {
        return err
    }
    server, err := ec2.NewInstance(ctx, "web-server", &ec2.InstanceArgs{
        Ami: pulumi.String("ami-0319ef1a70c93d5c8"),
            InstanceType:        pulumi.String("t2.micro"),
            VpcSecurityGroupIds: pulumi.StringArray{group.ID()},
        })
        if err != nil {
            return err
        }

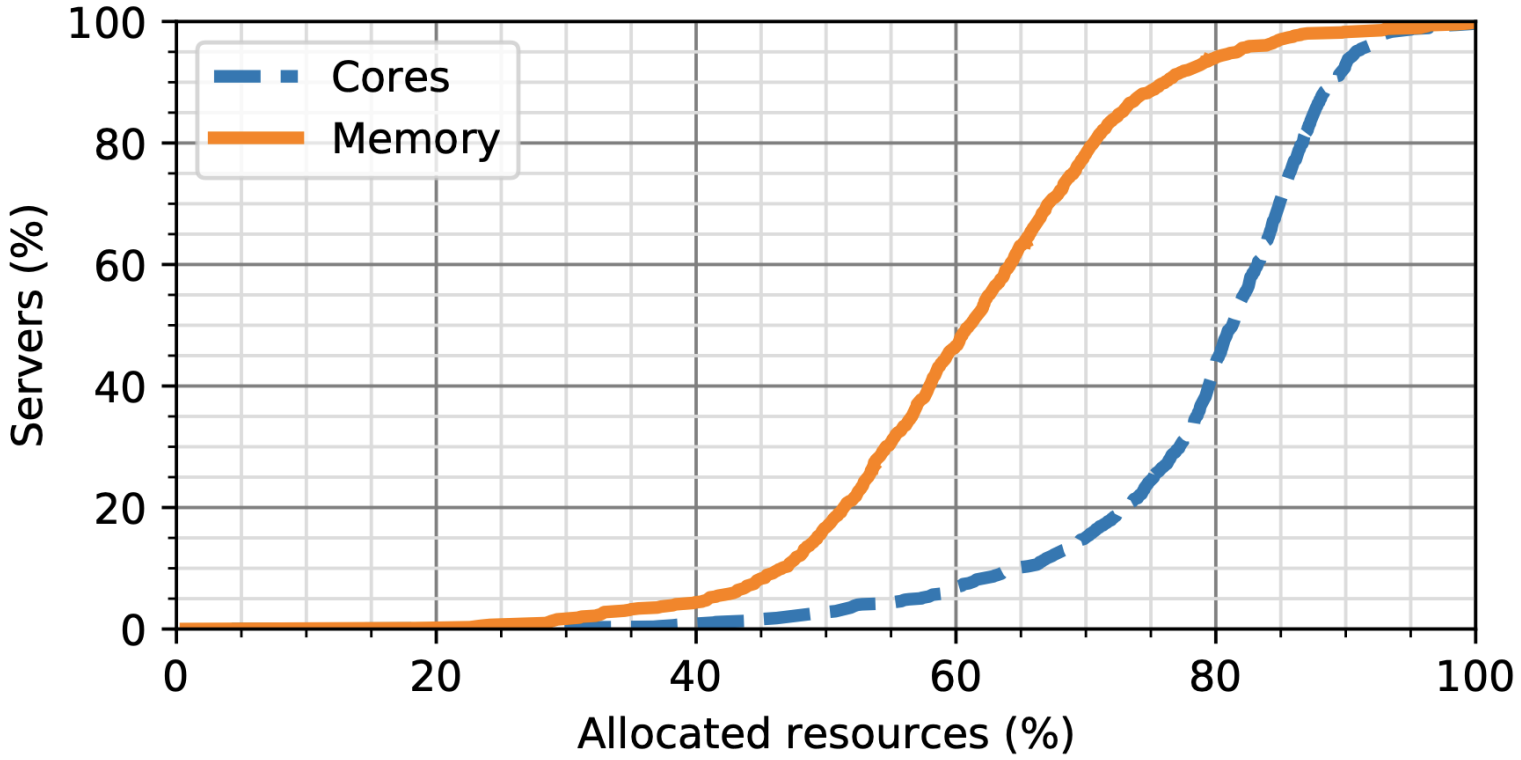
        ctx.Export("publicIp", server.PublicIp)
        ctx.Export("publicHostName", server.PublicDns)
        return nil
    })
}
```

Deploying applications on OpenStack

1. **Provision a VM**
   Example: 2 vCPUs, 2 GiB RAM, 20 GiB disk ← We need to decide hardware spec at deploy time
2. **Choose Ubuntu 22.04 image**
3. **Access the VM**
   SSH into the instance using key pair
4. **Install dependencies**
   Update system packages
   Install Apache/Nginx, MySQL/MariaDB, PHP (LAMP/LEMP stack)
5. **Deploy WordPress**
   Download WordPress package
   Configure database & credentials
   Place files in web root
6. **Access the application**
   Visit <VM_Floating_IP> in browser
   Complete WordPress setup wizard

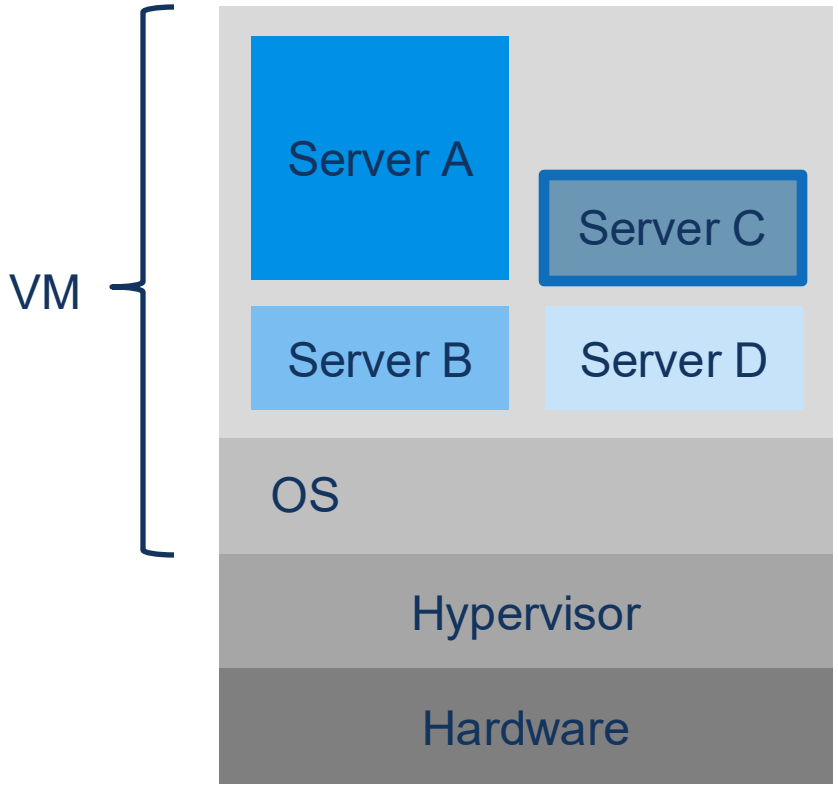
Challenge 1 - Low Resource Allocation

More than half of the servers (Azure) have an allocation of cores above 80%, while memory allocation is only between 50% and 70% for most servers

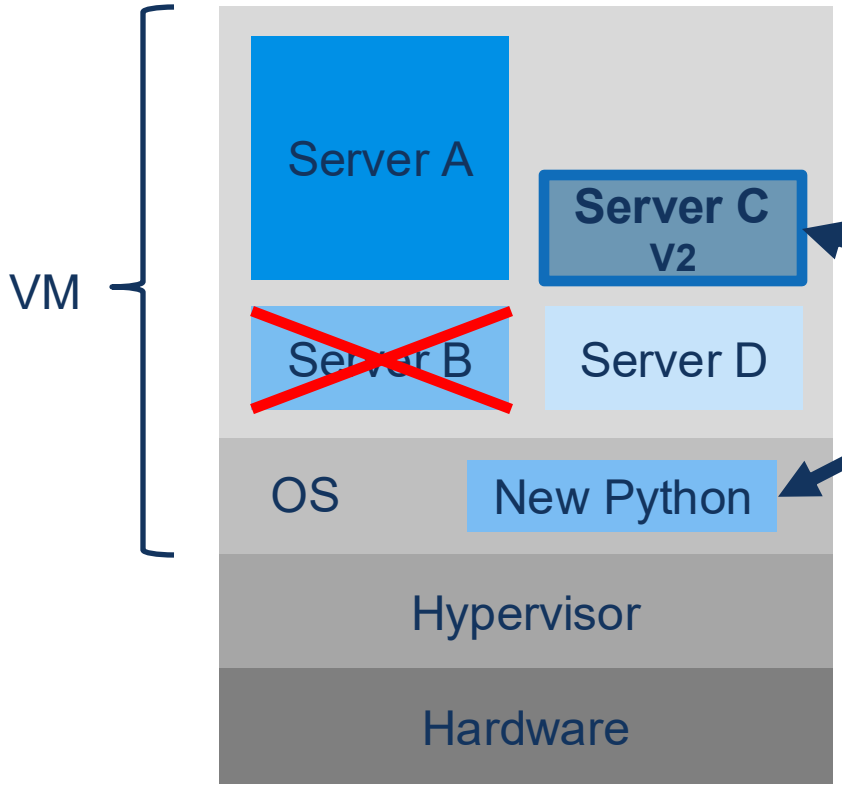


Source: A. Fuerst et al. 2022. *Memory-harvesting VMs in cloud platforms*. In Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22), https://doi.org/10.1145/3503222.3507725

Challenge 2 - Dependency Hell



Challenge 2 - Dependency Hell



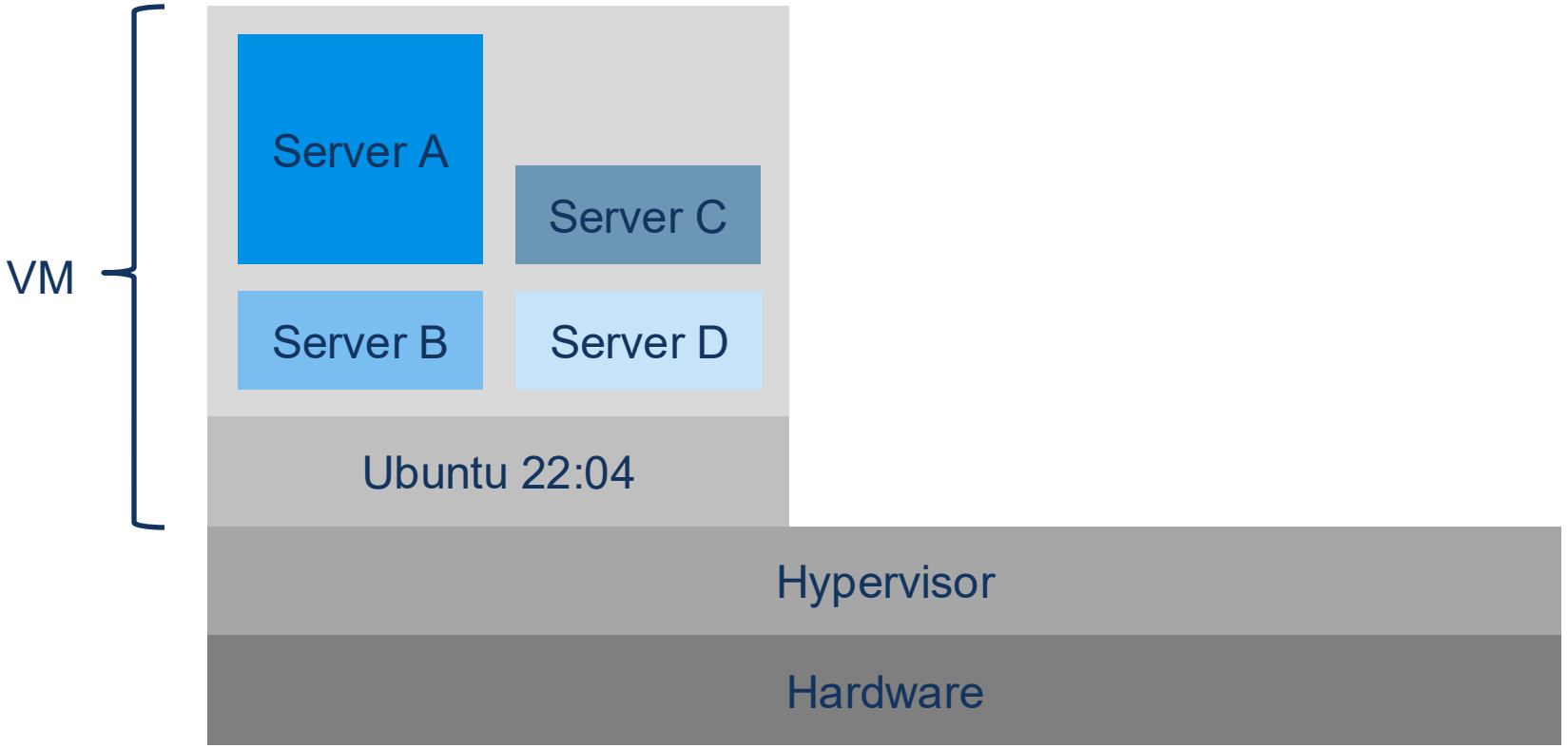
"Someone" updates Server C to V2, but then Server B crashes ☹

Several persons might need to collaborate install update a server

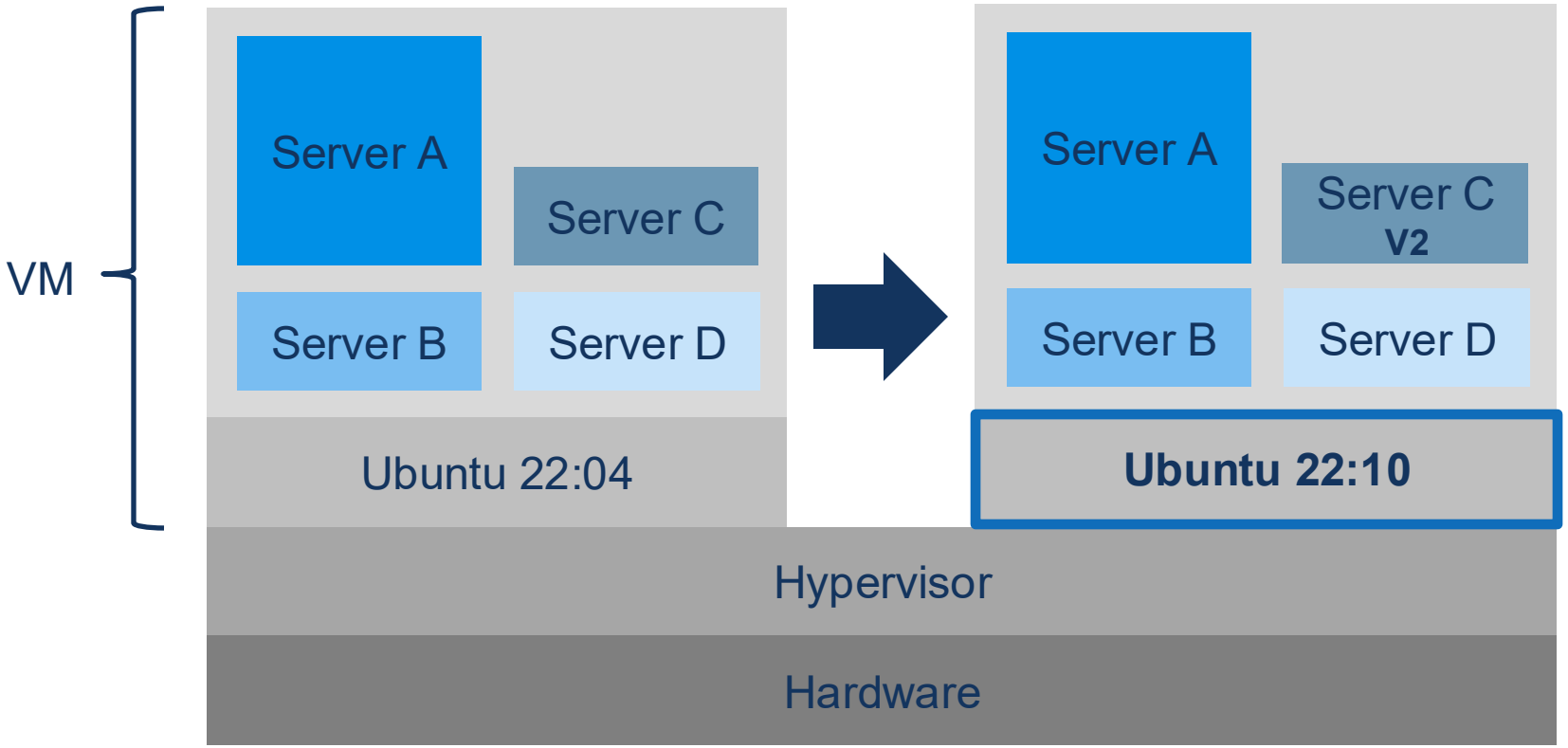
How to replicate the prod environment?

Challenge 2 - Dependency Hell

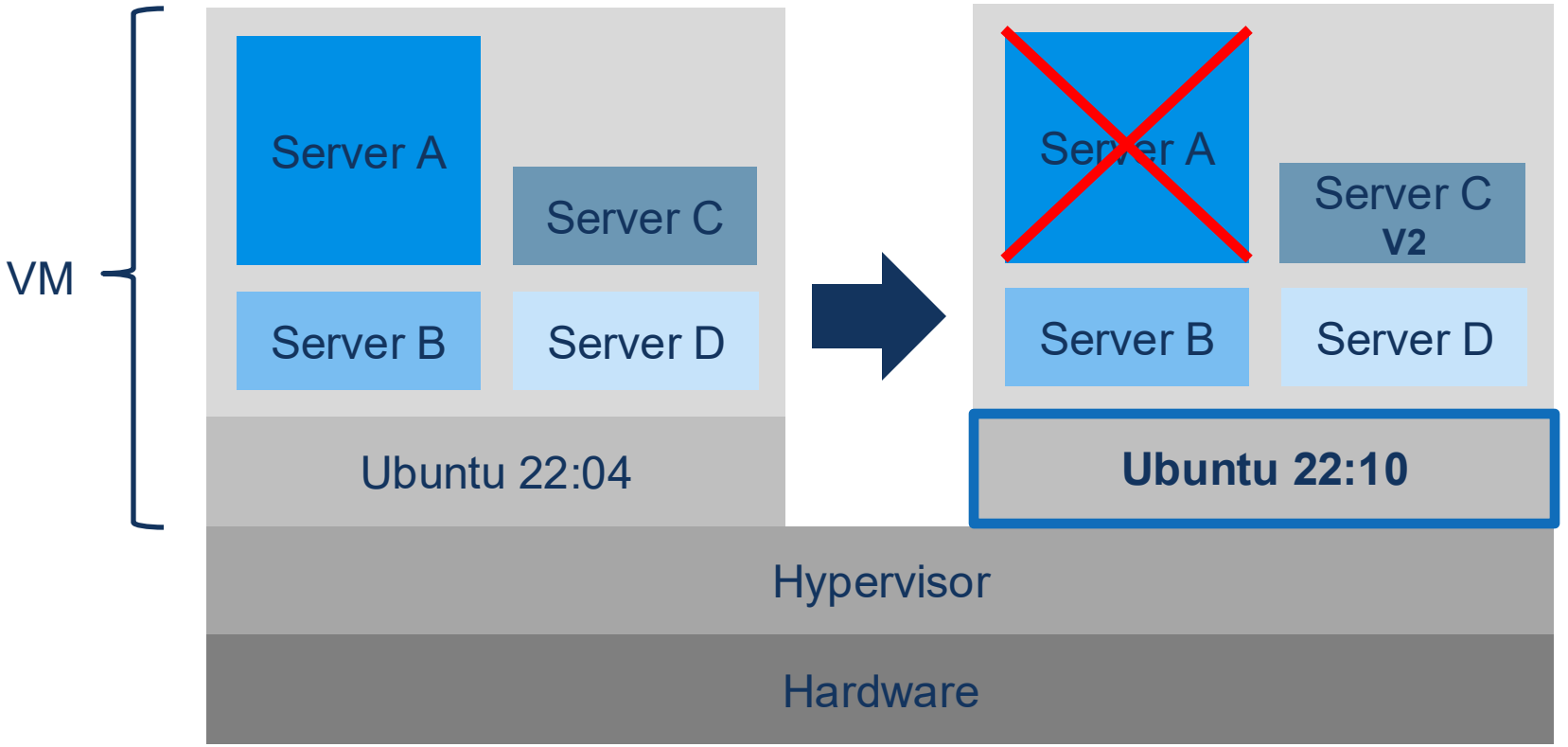
OS updates could cause severe problems!



Challenge 2 - Dependency Hell



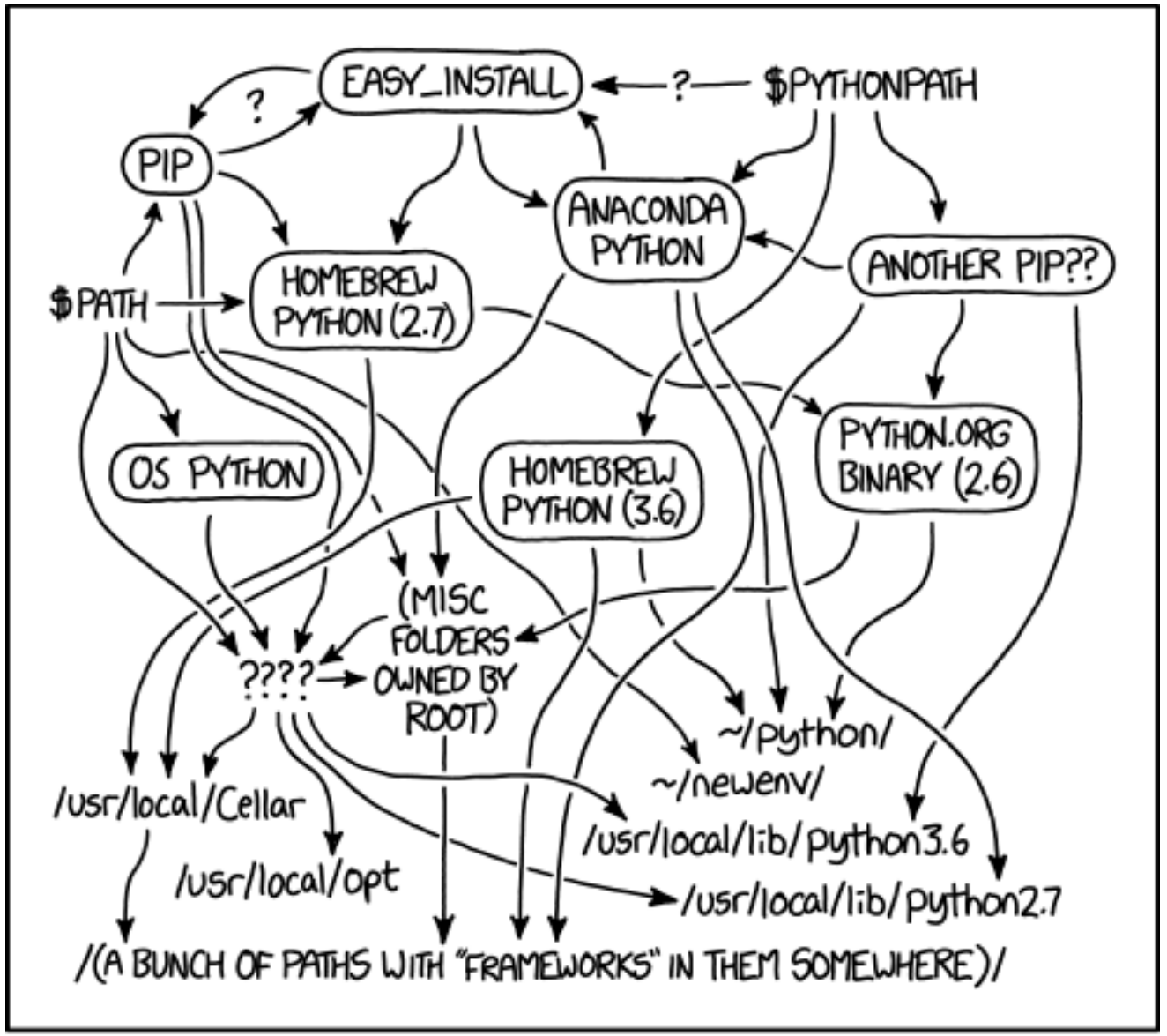
Challenge 2 - Dependency Hell



Server C V2 require Ubuntu 22:10

BUT

Server A only runs on Ubuntu 22:04

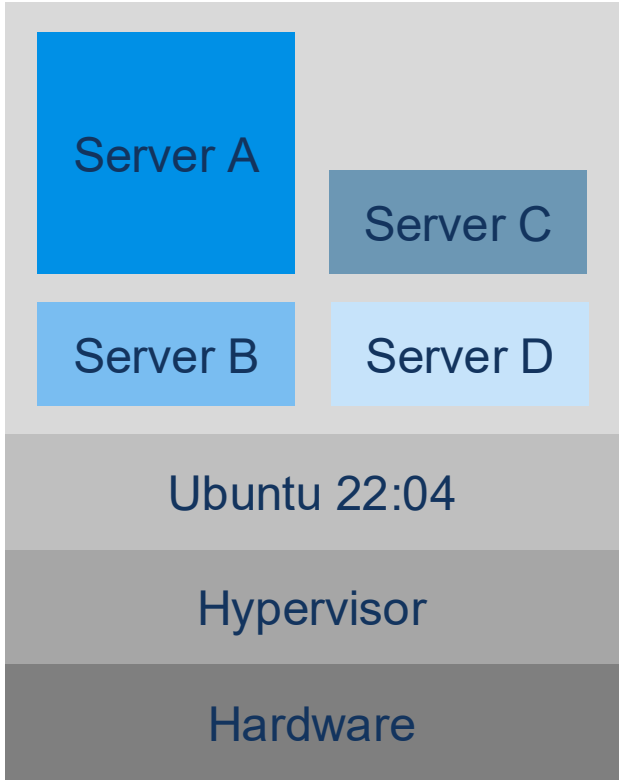


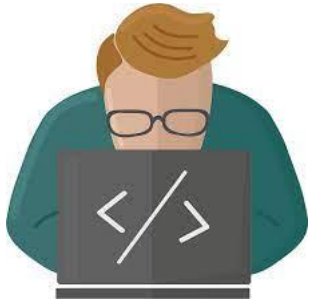
MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

Challenge 3 - Reproducibility

Each developer might have a slightly different local setup, leading to painful inconsistencies when code moved to test or production environments

Dev and prod environment are different

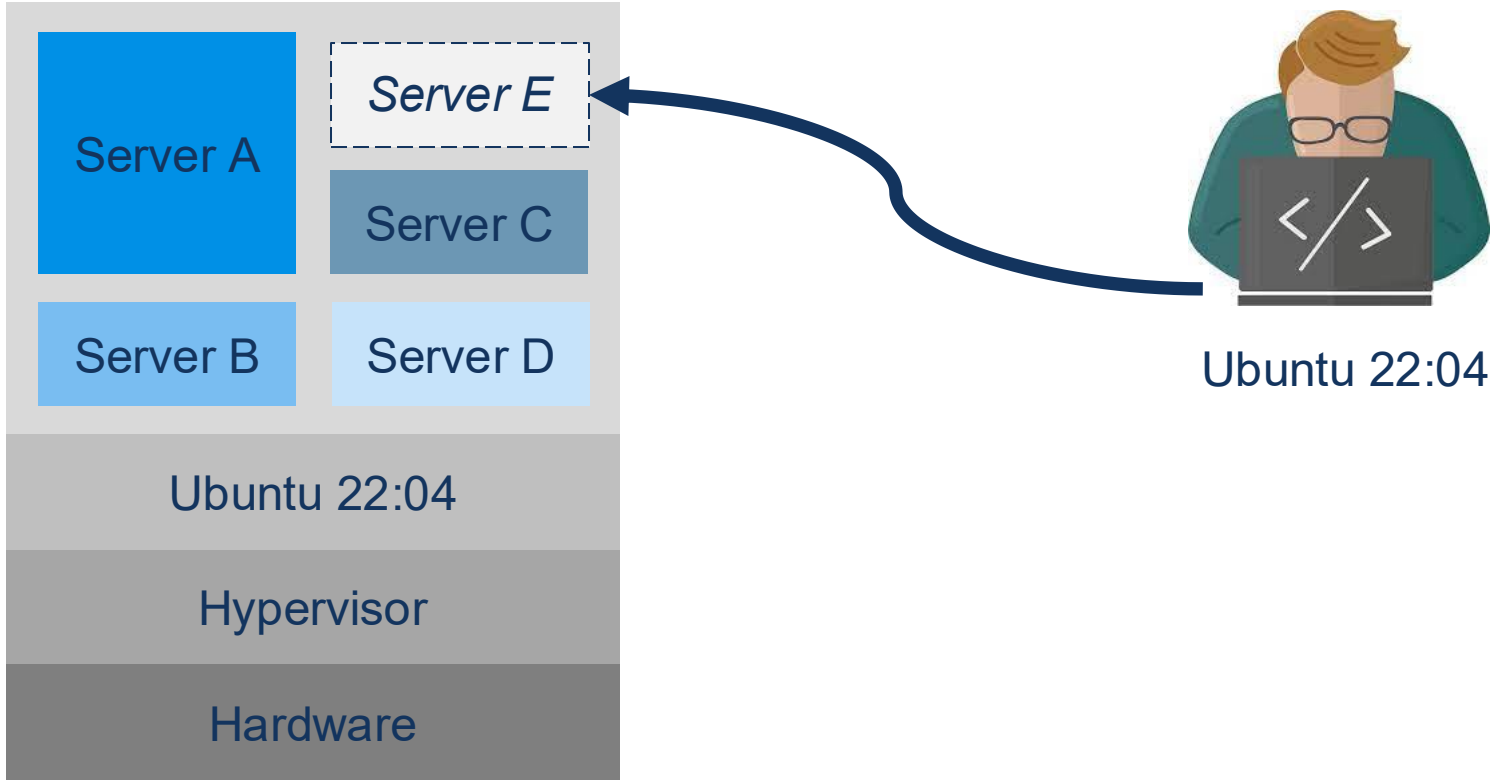




Ubuntu 22:04

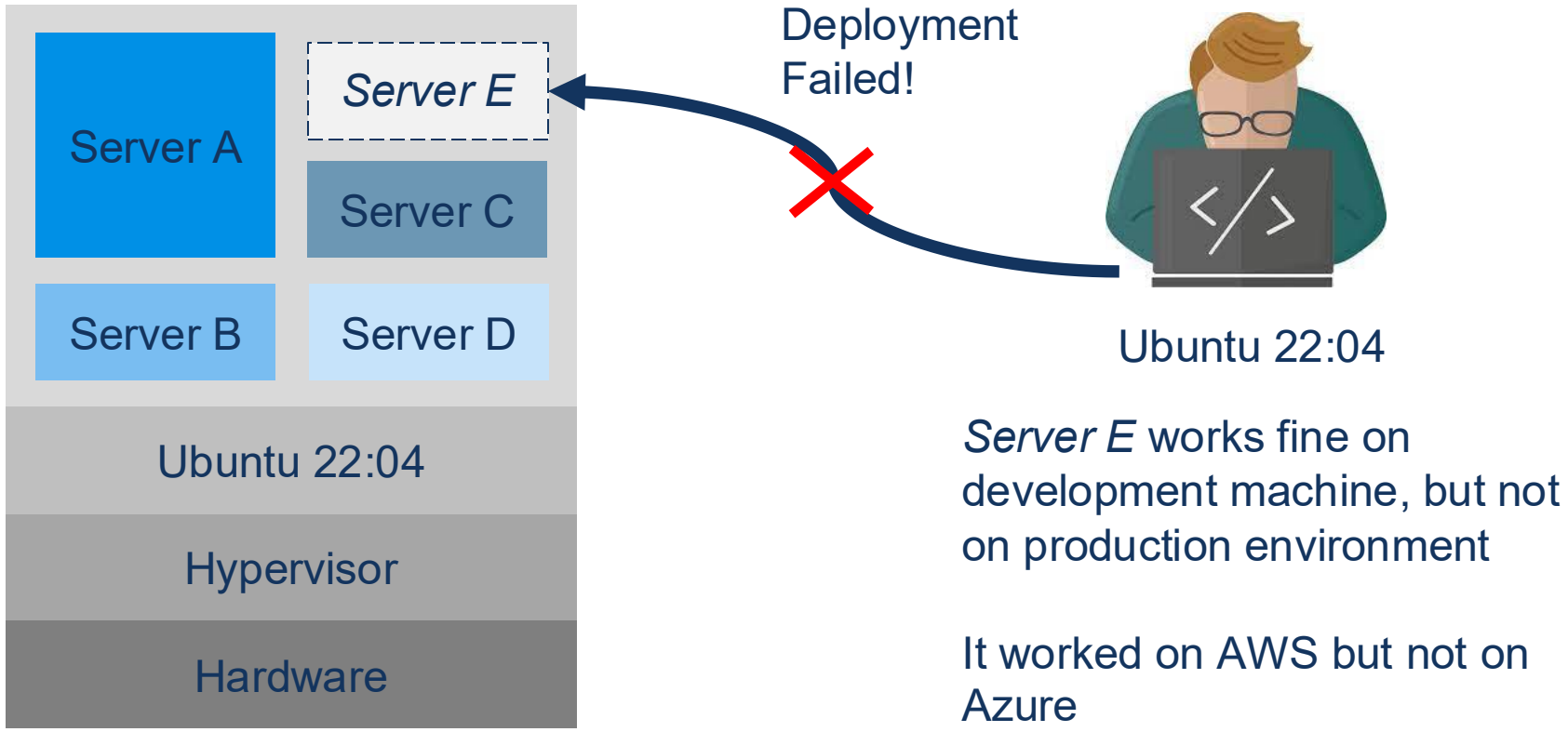
Challenge 3 - Reproducibility

Dev and prod environment are different



Challenge 3 - Reproducibility

Dev and prod environment are different



Server E works fine on development machine, but not on production environment

It worked on AWS but not on Azure

https://developer.hashicorp.com/vagrant

Vagrant



Vagrant was developed (around 2010) to address the ***it works on my machine*** syndrome

- Standardizing development environments, everyone works inside identical VMs defined by code.
- Making VMs disposable and repeatable, you can spin them up or destroy them in minutes.
- Bridging development and operations, by using the same provisioning tools (like Ansible or Puppet) that Ops teams use for servers
- Preecursor to modern container workflows, before Docker became popular,



Problems with Virtual Machines

Resource Overhead

- **Heavy VMs**: Each VM required a full OS installation (4-16GB per VM)
- **Memory waste**: Running 50 services meant 50 copies of Windows/Linux in memory
- **CPU overhead**: Hypervisor layer consumed 5-10% of CPU resources
- **Storage bloat**: VM images were large (20-100GB), making storage expensive

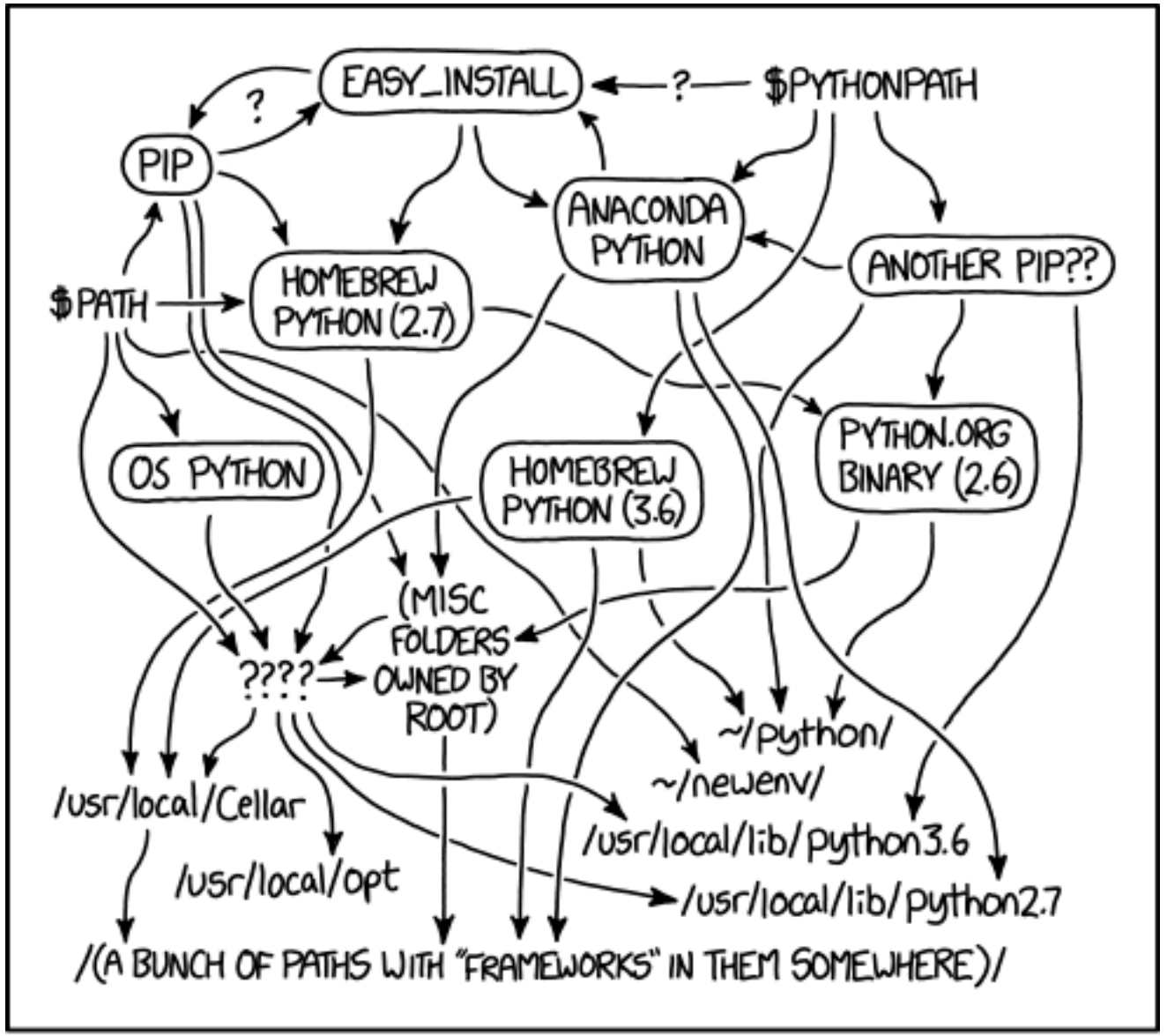
Performance Issues

- **Slow boot times:** VMs took 2-5 minutes to start up
- **I/O bottlenecks:** Virtual disk and network operations were slower than bare metal
- **Hypervisor overhead:** Extra layer of abstraction impacted performance

Problems with Virtual Machines

Management Complexity

- **VM sprawl**: Easy creation led to thousands of untracked VMs
- **Configuration drift**: VMs modified over time, losing consistency
- **Zombie VMs**: Forgotten VMs consuming resources with no clear owner
- **Backup complexity**: Backing up entire VMs was slow and storage-intensive
- **Update hell**: Patching OS in hundreds of VMs individually
- **Monolithic applications**: Apps weren't redesigned for virtualisation benefits



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.



- Loading/unloading was very time-consuming
- Inefficient transportation and storage
- Limited scalability
- Complex coordination
- Risk of damage and theft





“The storage itself needed to be transportable, not the goods within it”
In 1956 Malcolm McLean invented the shipping container



Container Era (2013 - Present)

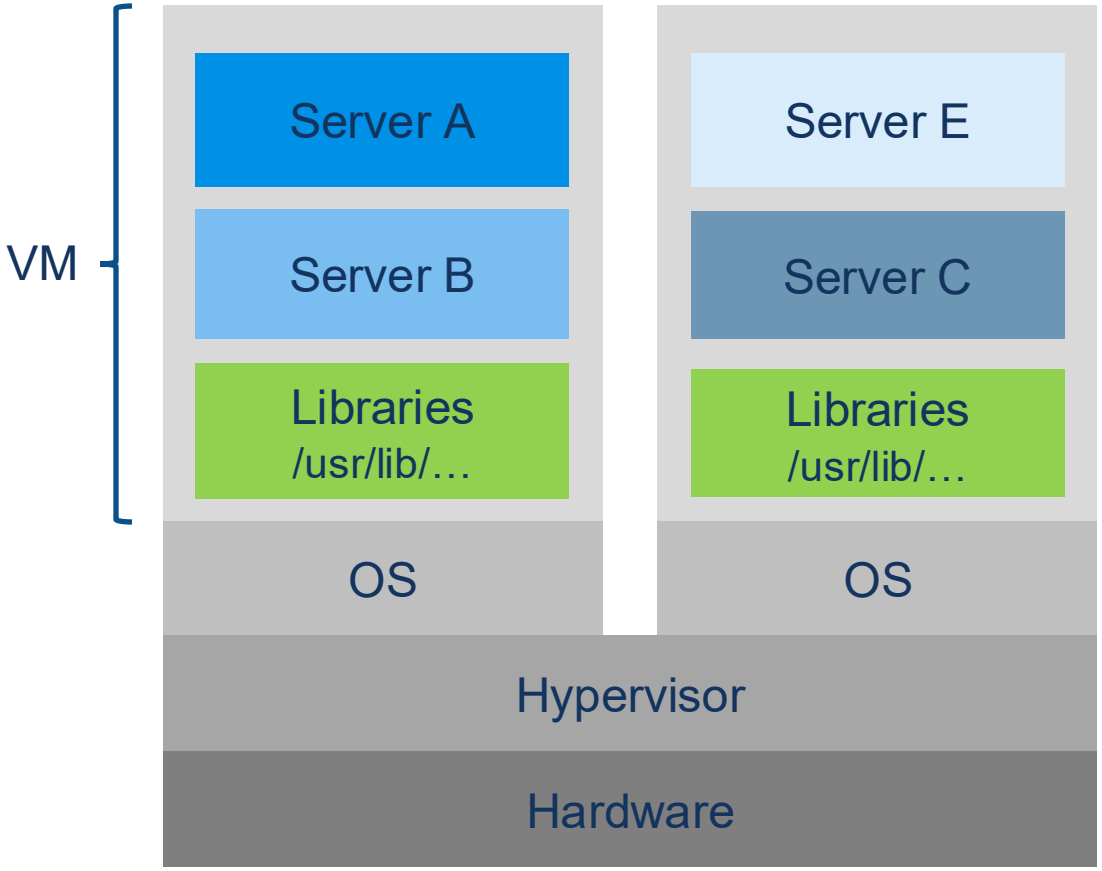
Docker container is a shipping container for code



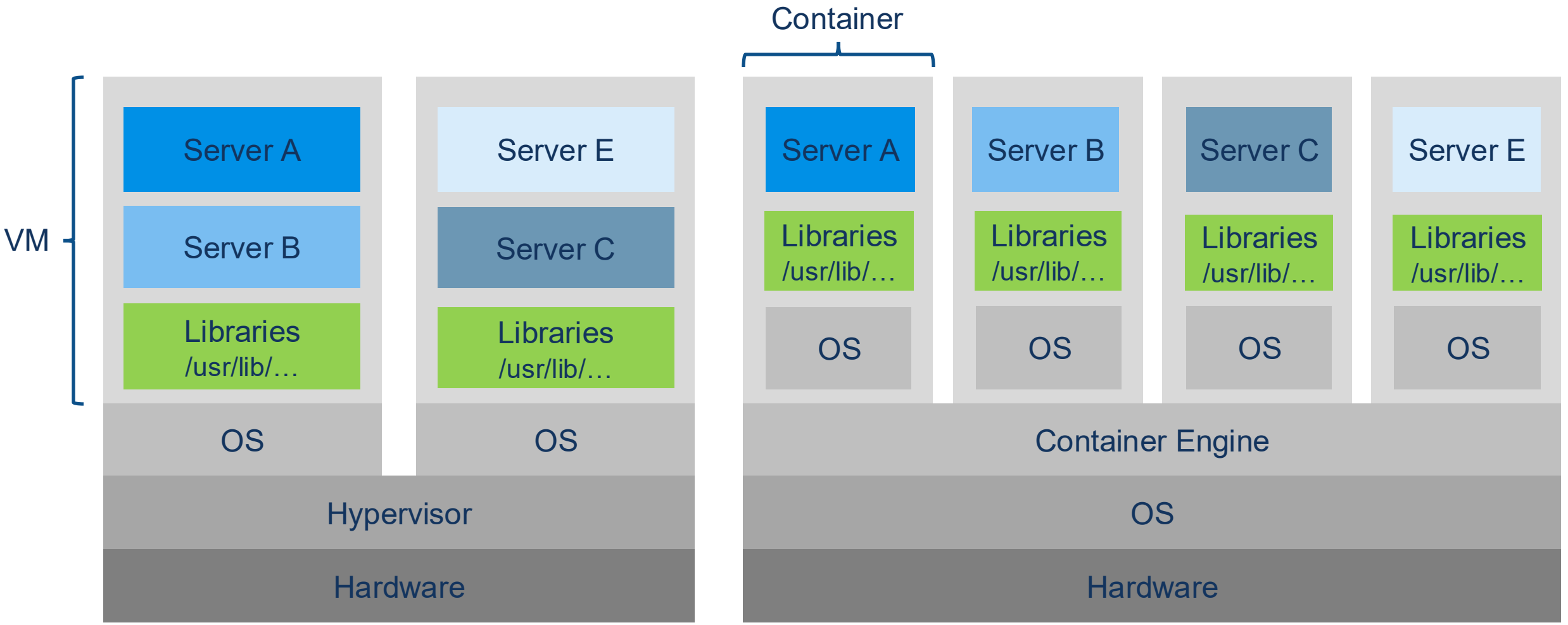
Docker was released in 2013

Containers package applications with their dependencies but share the host OS kernel, making them much lighter than VMs

Docker Containers



Docker Containers



Container images

- Packages an application and everything it needs files (code, libraries, runtime, config, dependencies etc.)
- Built in layers, i.e. efficient storage and re-use
- Immutable: every container started from the same image is identical
- Enables fast, reproducible deployments across dev, test, and production
- Stored in a registry (e.g. Docker Hub, Harbor, Quay) and pulled on demand

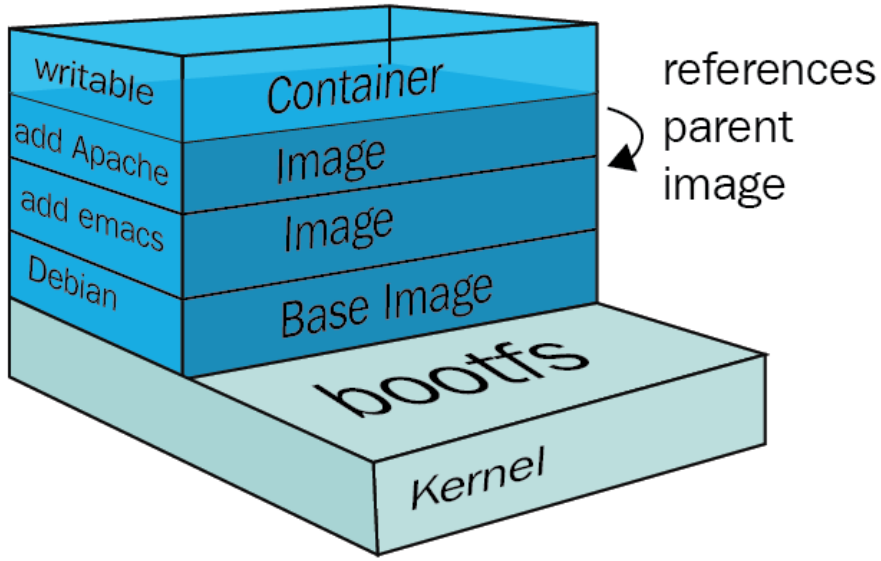
```
FROM python:3.13
WORKDIR /usr/local/app

# Install the application dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

# Copy in the source code
COPY src ./src
EXPOSE 8080

# Setup an app user so the container doesn't run as the root user
RUN useradd app
USER app

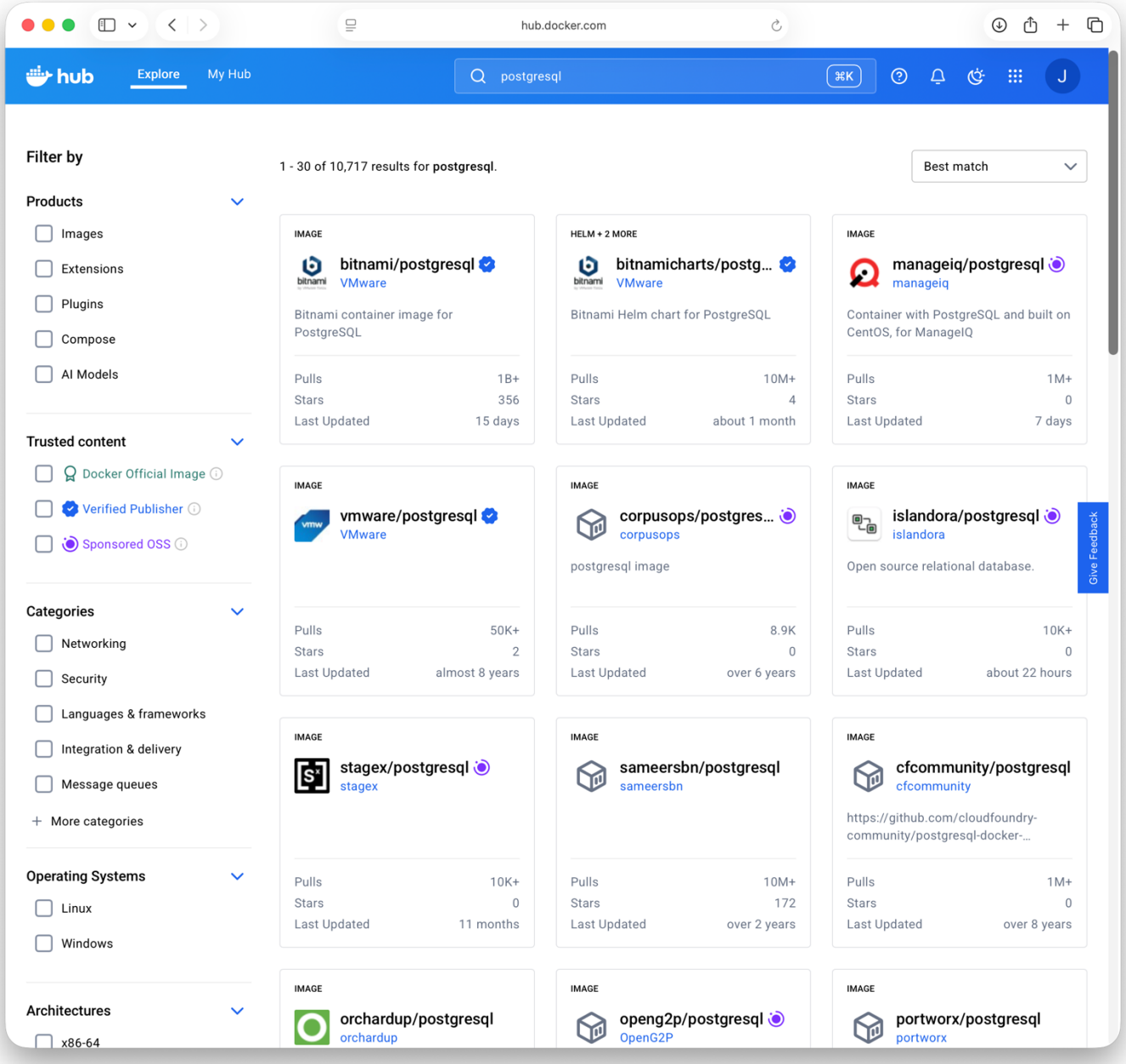
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```



Docker Registry

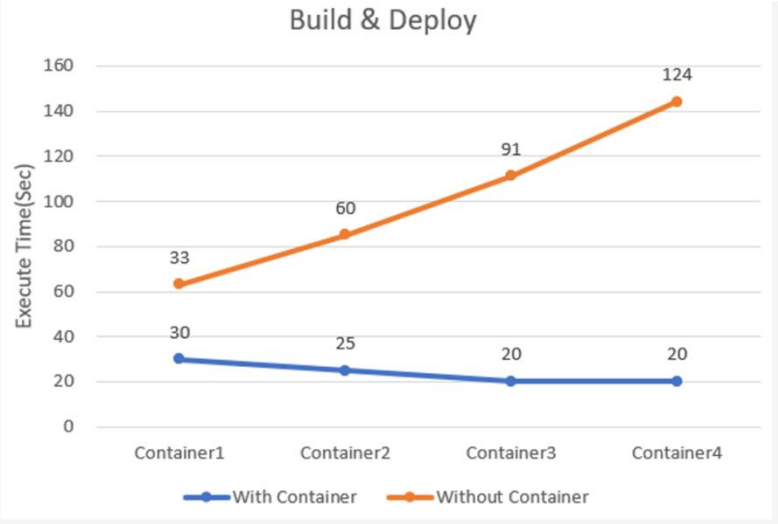
$ docker push <image>

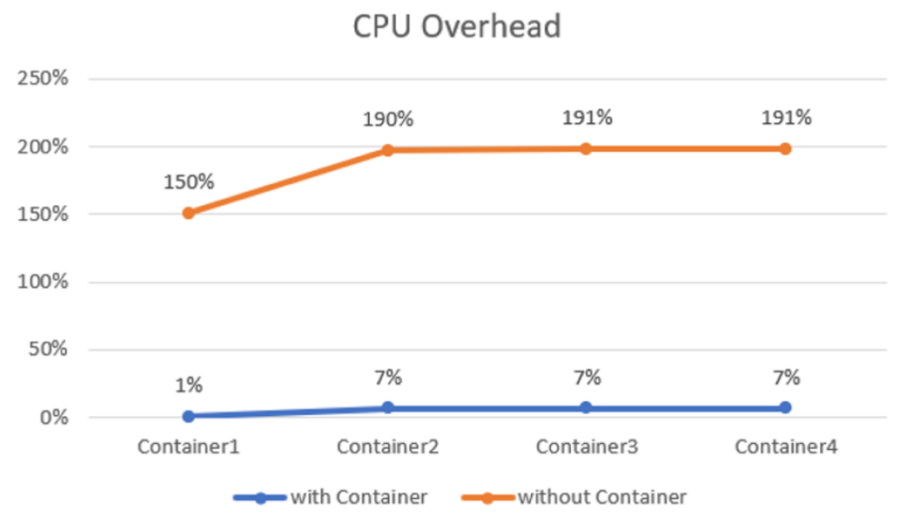
$ docker pull <image>

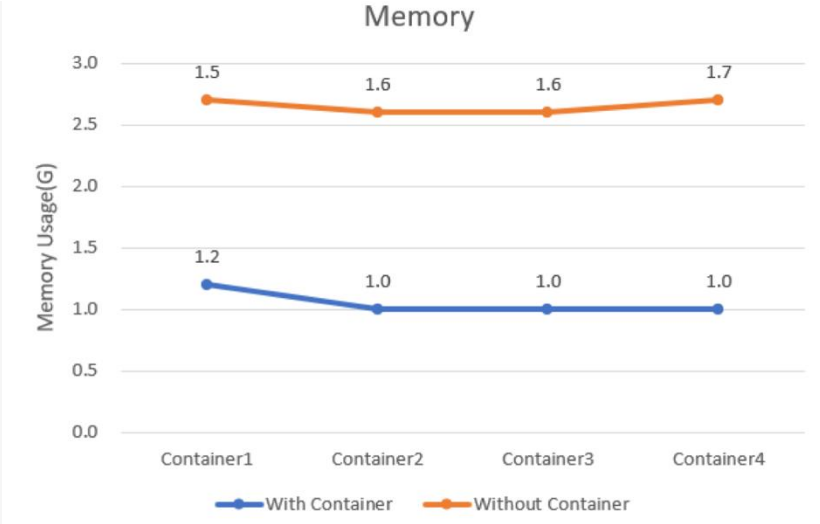


Containers vs VMs

Docker's lightweight containerisation architecture significantly reduces CPU overhead and improves scalability compared to traditional virtualisation







Source: Kim BS, Lee SH, Lee YR, Park YH, Jeong J. *Design and Implementation of Cloud Docker Application Architecture Based on Machine Learning in Container Management for Smart Manufacturing*. Applied Sciences. 2022; 12(13):6737. https://doi.org/10.3390/app12136737

Container Era (2013 - Present)

Improvements

- **Lightweight**: Containers start in seconds, use MB instead of GB
- **Density**: Run 100s-1000s of containers per host (vs 10s of VMs)
- **Portability**: "Build once, run anywhere" - dev laptop to production

New Paradigms

- **Immutable Infrastructure**: Containers are disposable, never patched
- **Declarative Configuration**: Describe desired state, system maintains it
- **Service Mesh**: Advanced networking between microservices (Istio, Linkerd)
- **DevOps/GitOps**: Git as source of truth for infrastructure
- **Cloud Native Apps**: Applications (microservices) designed for container orchestration

Linux Namespaces

Namespaces have been part of the Linux kernel since about 2002 and isolate processes from each other

- PID Namespace: Isolates process IDs and process trees
- Mount Namespace: Isolates filesystem mount points and views
- Network Namespace: Isolates network interfaces, routing tables, firewall rules
- UTS Namespace: Isolates hostname and domain name
- IPC Namespace: Isolates inter-process communication resources
- User Namespace: Isolates user and group ID mappings

Namespace Example

```
# List processes
ps aux | head -10

# Check your current PID
echo "My PID: $$"

# View process tree
pstree -p | head -20

# Note the init process (PID 1)
ps -p 1
```

unshare - run program in new namespaces

```
# This isolates both PID and mount namespaces
sudo unshare -p -m --fork /bin/bash

# Now remount /proc (won't affect host)
mount -t proc proc /proc

# Check your PID in the new namespace
echo "My PID in namespace: $$"

# Try to view processes
ps aux

# Check what PID 1 is
ps -p 1

# View process tree in namespace
pstree -p
```

Namespace Full Example

```
sudo unshare -p -m --fork /bin/bash -c '
    mount -t proc proc /proc &&
    mount -t tmpfs tmpfs /mnt &&
    mkdir -p /mnt/{usr,lib,lib64,proc,tmp,dev,bin} &&
    mount --bind /usr /mnt/usr &&
    mount --bind /lib /mnt/lib &&
    mount --bind /lib64 /mnt/lib64 &&
    mount --bind /bin /mnt/bin &&
    mount -t proc proc /mnt/proc &&
    mknod /mnt/dev/null c 1 3 &&
    chroot /mnt /bin/bash
'
```

Cgroup

- namespaces = ***What can you see?***
  - Control what processes can see and access
  - Provide separate views of system resources
- cgroups = ***How much?***
  - Limit CPU, memory, disk I/O usage
  - Control resource allocation and prioritization
- cgroups and namespaces are two complementary Linux kernel features that work together to provide complete container isolation
- cgroups ensure the container can't use more than its allocated resources, while namespaces ensure it can't see or interfere with other containers or the host system

Cgroup Example

The tee command reads from input and writes to both output AND a file simultaneously

Let’s create a cgroup with MAX 50M memory usage

```
echo '+memory' | sudo tee /sys/fs/cgroup/cgroup.subtree_control >/dev/null
echo 50M | sudo tee /sys/fs/cgroup/demo/memory.max >/dev/null
echo 0 | sudo tee /sys/fs/cgroup/demo/memory.swap.max >/dev/null
```

Cgroup Example

```
sudo unshare --pid --net --uts --fork --mount-proc bash -c ‘
    echo $$ | tee /sys/fs/cgroup/demo/cgroup.procs >/dev/null
    python3 -c ”
import mmap, os
print(f\"PID: {os.getpid()}\")
print(\"cgroup:\", open(\"/proc/self/cgroup\").read().strip())

try:
    # Allocate 100MB and WRITE to every page
    size = 100 * 1024 * 1024
    data = mmap.mmap(-1, size)

    for i in range(0, size, 4096):
        data[i] = 42
        if i % (10 * 1024 * 1024) == 0:
            print(f\"Touched {i // (1024*1024)}MB\")
        print(\"ERROR: Should not reach here!\")

except Exception as e:
print(f\"Hit limit: {e}\")

"

' || echo "Process was OOM killed - memory limit working!"
```



Create a namespace



Add process to Cgroup



Try to allocate 100MB

What is a Container?

Container = namespaces + cgroups + rootfs + runtime

namespaces = *What can you see?*

- PID: Separate process tree
- Network: Isolated network stack
- Mount: Separate filesystem view
- UTS: Different hostname
- User: Different user/group mappings
- IPC: Isolated inter-process communication

cgroups = *How much can you use?*

- Memory: RAM limits
- CPU: CPU time limits
- I/O: Disk bandwidth limits
- PIDs: Process count limits

rootfs = *What filesystem do you see?*

- Container image (Ubuntu, Alpine, etc.)
- Separate from host filesystem

Container Runtime = *Who manages it all?*

- Docker, Podman, containerd, Singularity
- Orchestrates namespaces + cgroups + filesystem

Container VS Virtual Machines

	Hypervisor (VMs)	Namespaces + cgroups (Containers)
Kernel	Separate kernel per VM	Shared kernel
Memory	Hardware EPT/NPT translation	Software cgroup limits
CPU	Virtual CPUs, hardware scheduling	Process scheduling, cgroup quotas
Storage	Virtual disks, device emulation	Bind mounts, overlay filesystems
Network	Virtual NICs, software switches	Virtual interfaces, bridges
Security	Hardware isolation boundary	Software isolation boundary
Boot time	Seconds to minutes	Milliseconds
Resource overhead	High (full OS per VM)	Low (shared kernel)
Isolation level	Complete (different OS)	Process-level (same OS)

Alternative Container Runtimes

Docker

The original and most popular runtime; includes both the container engine and tools for building, distributing, and managing containers

Podman

Docker-compatible CLI, daemonless (runs as user)

Containerd

Core runtime used by Docker and Kubernetes; handles container lifecycle only

Apptainer/Singularity (HPC)

Designed for High-Performance Computing, focuses on reproducible, secure scientific workloads without requiring root privileges

Kata container

Combines VM-level isolation (using lightweight microVMs) with container APIs (security focus)

MicroVMs

Firecracker

Firecracker is a Virtual Machine Monitor (VMM) similar to QEMU

Based on KVM, but strips out everything not essential for cloud-scale, short-lived workloads

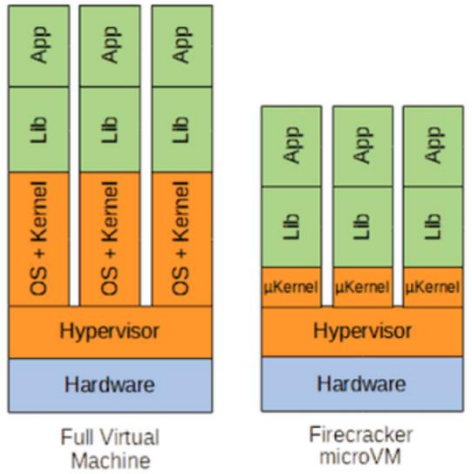
- No BIOS/UEFI, No Grub, PCI, USB, GPU, audio, etc.

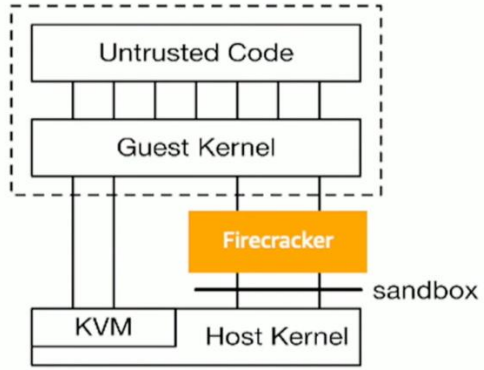
Gives strong isolation like traditional VMs (each guest has its own kernel, memory, and CPU space), but approaches the startup speed and footprint of containers

Build for AWS Lambda and AWS Fargate to overcome that containers are too weakly isolated for multi-tenant clouds and normal VMs are too slow to start

Used by Kata containers (https://katacontainers.io)

- Open-source container runtime combining virtual machines (VMs) and containers for stronger isolation
- Runs each container inside a lightweight microVM with its own kernel
- Compliant with the OCI (Open Container Initiative)

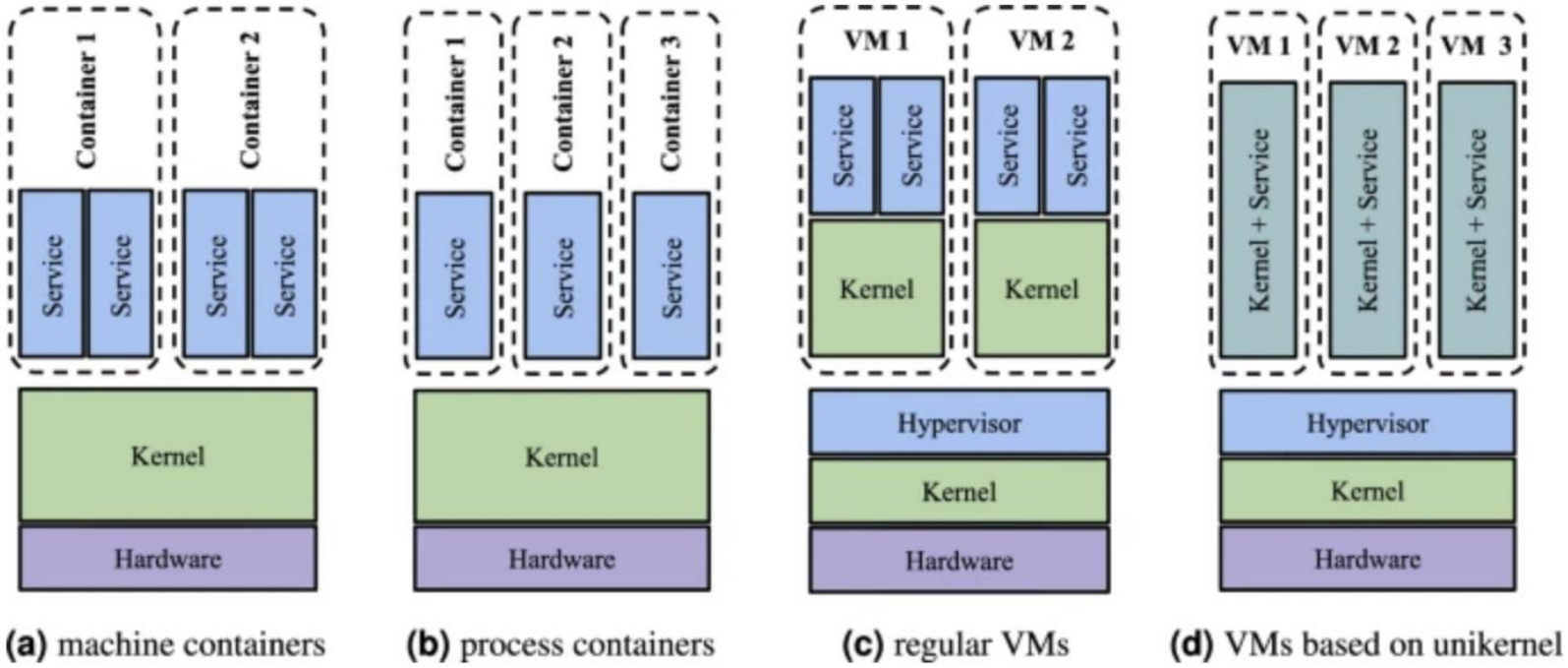




Agache, A., Brooker, M., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., & Popa, D.-M. (2020). *Firecracker: Lightweight virtualization for serverless applications*. In Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20) (pp. 419–434). USENIX Association

Unikernels

A ***unikernel*** is a highly specialized, single-purpose operating system image that combines just the minimal OS components needed with an application, allowing it to run directly on a hypervisor or hardware with extreme efficiency and security



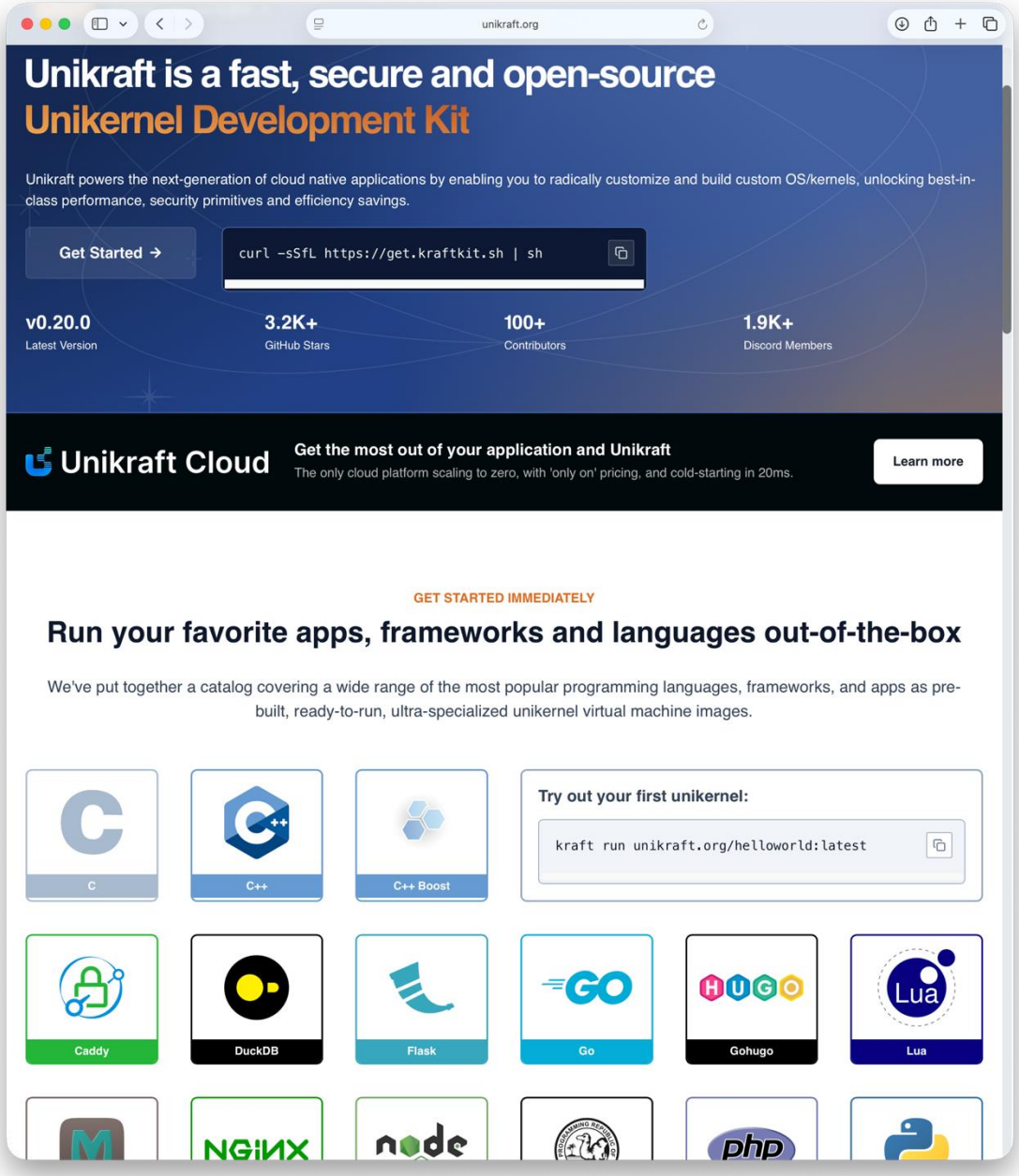
Source: Kurek, T., Niemiec, M. & Lason, A. Performance evaluation of a firewall service based on virtualized Include OS unikernels. Sci Rep 14, 557 (2024). https://doi.org/10.1038/s41598-024-51167-8

Unikraft

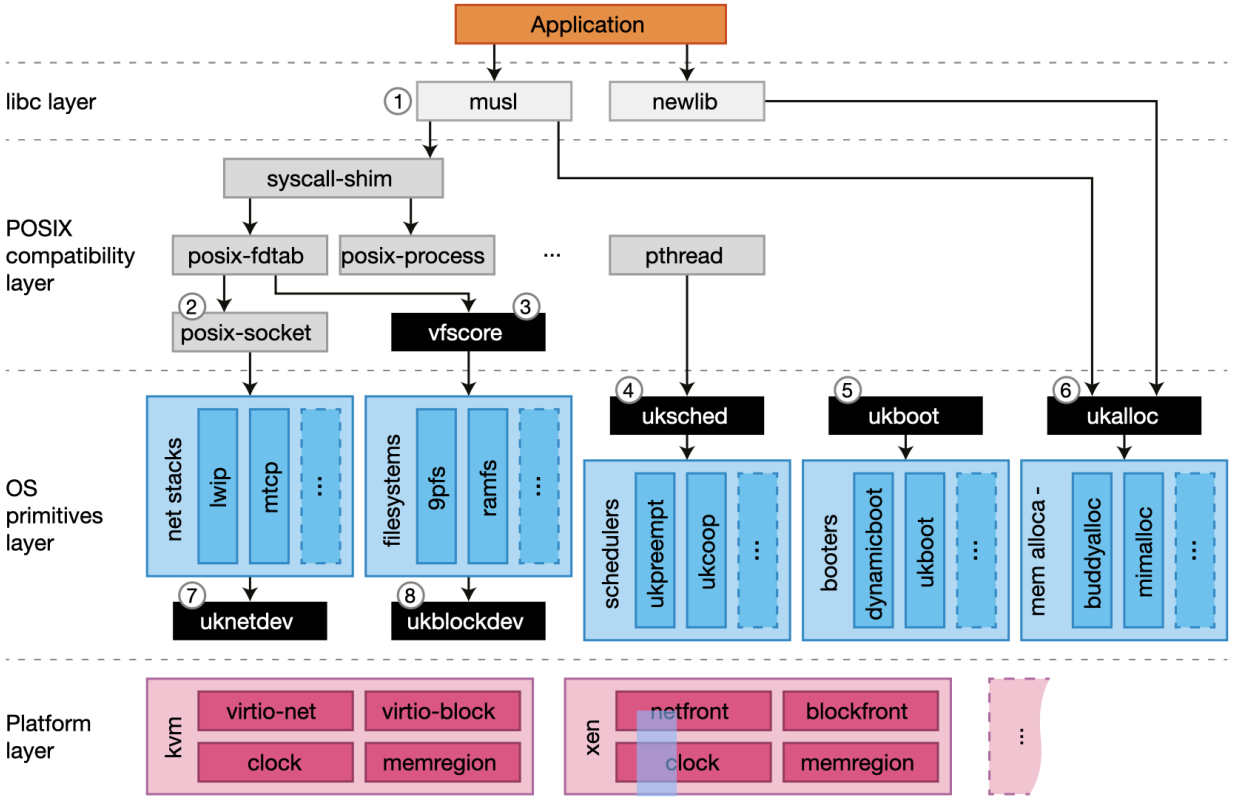
The high-level goal of Unikraft is to be able to build specialised OS images, known as ***unikernels***, easily, quickly and without time-consuming expert work

A key distinction between unikernels and classical monolithic Operating Systems (OS) is that it is a single-address space binary object

- It does not have separation between kernel and user address spaces which allows for much faster execution



Unikraft



https://github.com/unikraft/catalog/tree/main/example

Name	Last commit message	Last commit da...
..		
app	refactor(examples): Change directory names	5 months ago
.dockerignore	refactor(examples): Change directory names	5 months ago
.gitignore	refactor(examples): Change directory names	5 months ago
Dockerfile	refactor(examples): Change directory names	5 months ago
Kraftfile	refactor(examples): Change directory names	5 months ago
README.md	docs(examples): Update README titles for Node.js examples	last month
package.json	refactor(examples): Change directory names	5 months ago

README.md

Node 21 ExpressJS

This directory contains a Node 21 `ExpressJS` implementation running on Unikraft.

Set Up

To run this example, install Unikraft's companion command-line toolchain `kraft`, clone this repository and `cd` into this directory.

Run and Use

Use `kraft` to run the image and start a Unikraft instance:

```
kraft run --rm -p 3000:3000 --plat qemu --arch x86_64 -M 512M .
```

If the `--plat` argument is left out, it defaults to `qemu`. If the `--arch` argument is left out, it defaults to your system's CPU architecture.

Once executed, it will open port `3000` and wait for connections. To test it, you can use `curl`:

```
curl localhost
```

You should see a "Bye, World!" message.

Inspect and Close

To list information about the Unikraft instance, use:

```
kraft ps
```

```
NAME                    KERNEL                        ARGS                          CREATED         STATUS   M
agitated_davidgreybeard oci://unikraft.org/node:21    /usr/bin/node /usr/src/server.js  10 seconds ago  running  4
```

The instance name is `agitated_davidgreybeard`. To close the Unikraft instance, close the `kraft` process (e.g., via `Ctrl+c`) or run:

```
kraft rm agitated_davidgreybeard
```

Note that depending on how you modify this example your instance **may** need more memory to run. To do so, use the `kraft run`'s `-M` flag, for example:

```
kraft run -p 3000:3000 --plat qemu --arch x86_64 -M 1024M .
```

Unikraft Performance

Unikraft: Fast, specialized unikernels the easy way
Simon Kuenzer et al. – EuroSys 2021
DOI: 10.1145/3447786.3456248
https://dl.acm.org/doi/pdf/10.1145/3447786.3456248

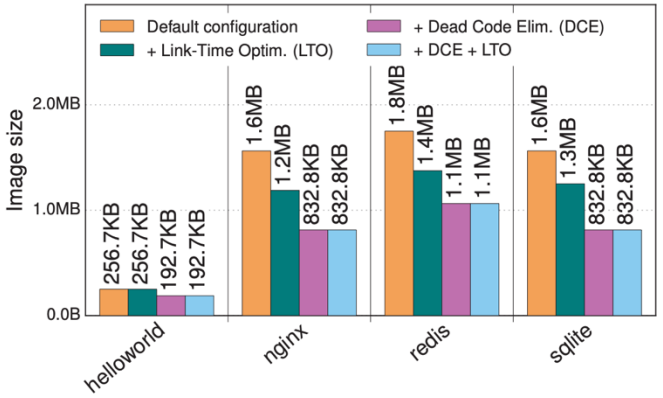


Figure 8. Image sizes of Unikraft applications with and without LTO and DCE.

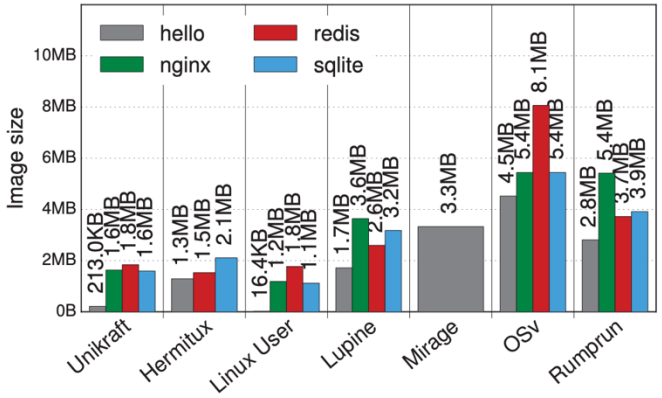


Figure 9. Image sizes for Unikraft and other OSes, stripped, w/o LTO and DCE.

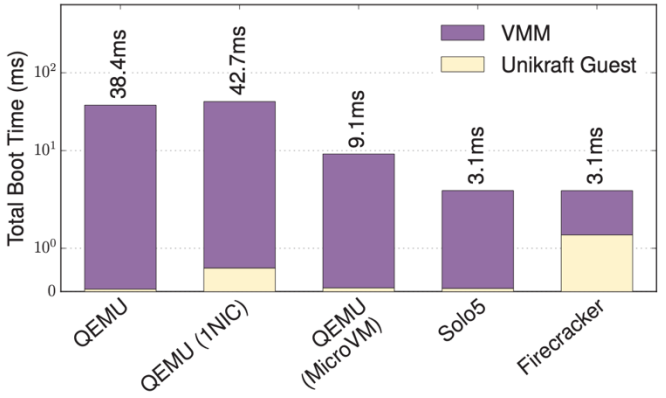


Figure 10. Boot time for Unikraft images with different virtual machine monitors.

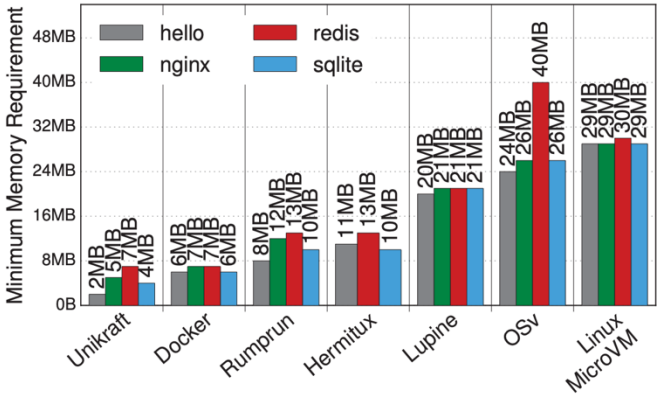


Figure 11. Minimum memory needed to run different applications using different OSes, including Unikraft.

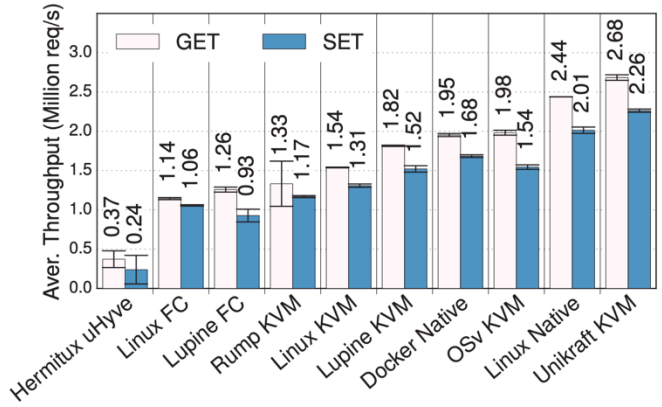


Figure 12. Redis perf (30 conns, 100k reqs, pipelining 16) with QEMU/KVM and Firecracker (FC).

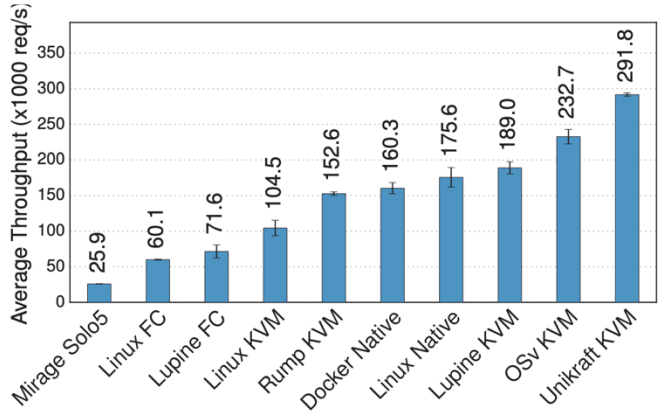
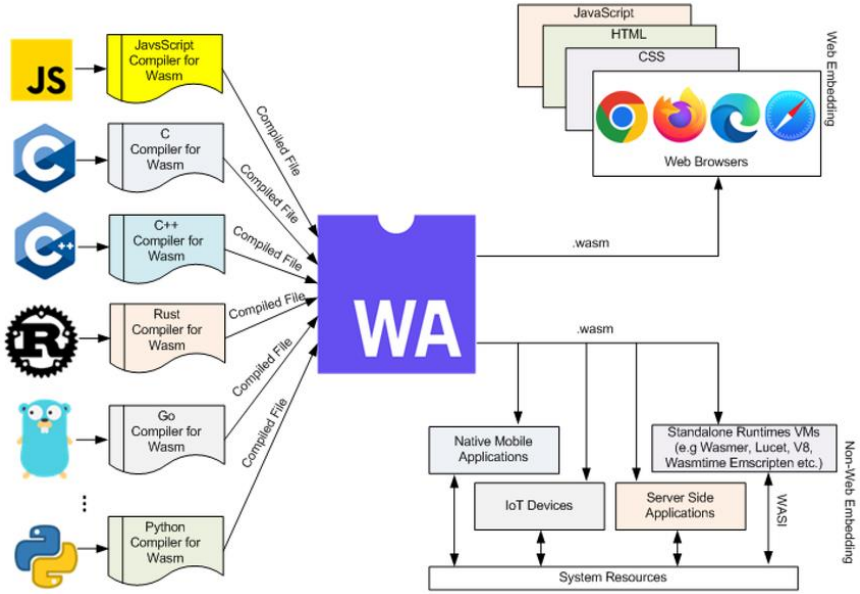
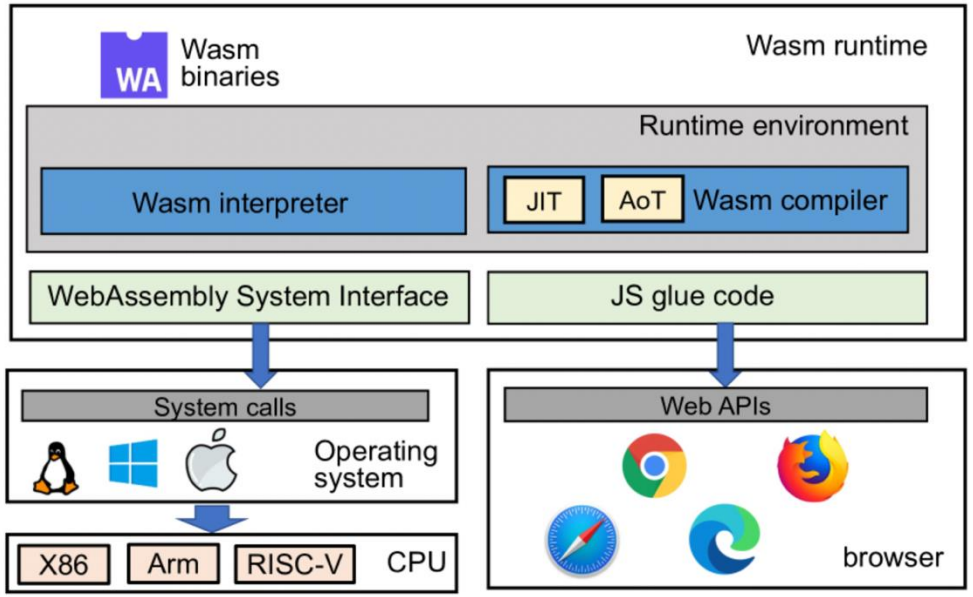


Figure 13. NGINX (and Mirage HTTP-reply) performance with `wrk` (1 minute, 14 threads, 30 conns, static 612B page).

WebAssembly

- Portable binary instruction format designed for the web
- Runs inside a secure sandboxed runtime (e.g., Wasmtime, WasmEdge)
- Language-agnostic: compile from C/C++, Rust, Go, AssemblyScript, etc.
- Extremely fast startup time (milliseconds) and small memory footprint
- Safety by default: memory-safe, no direct syscalls, capability-based access
- Ideal for serverless, plugins, edge compute, and lightweight isolated workloads



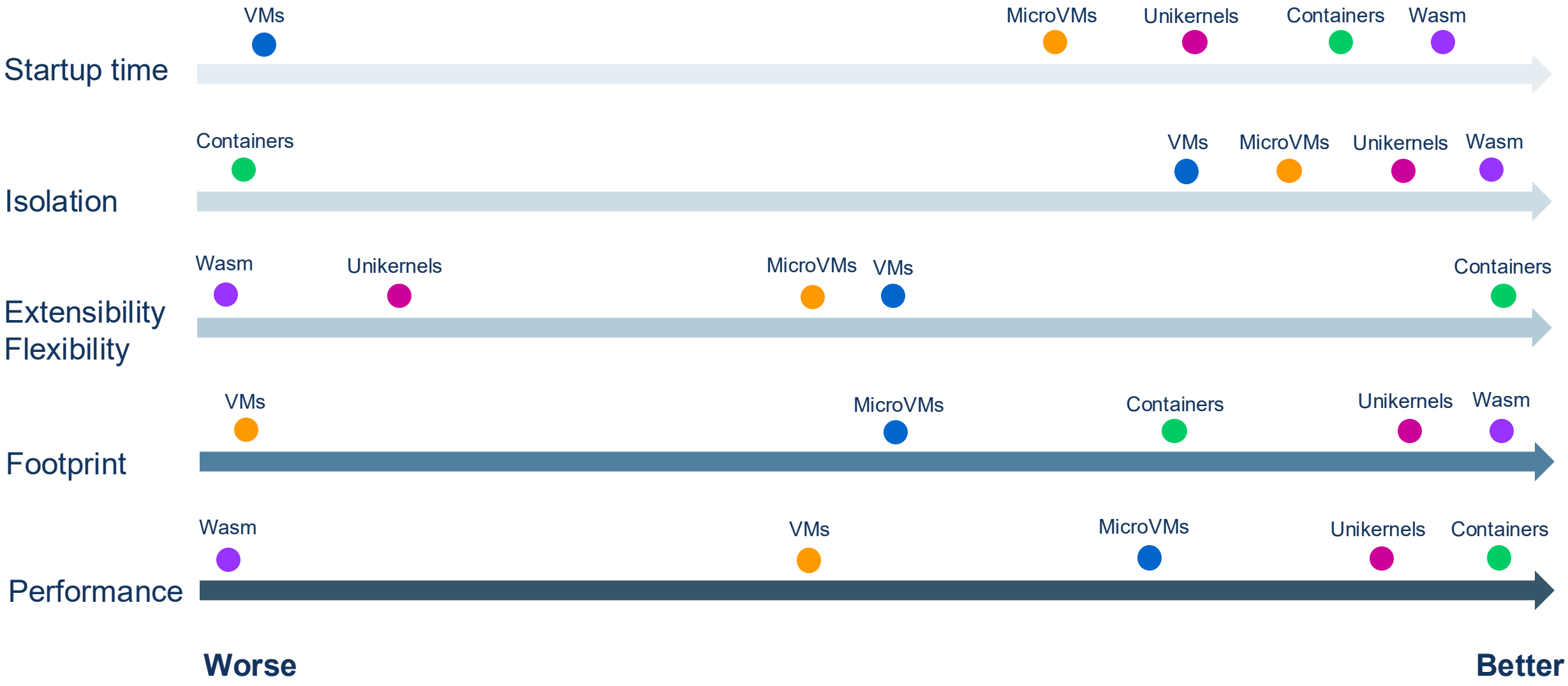


https://arxiv.org/html/2404.12621v1

Summary

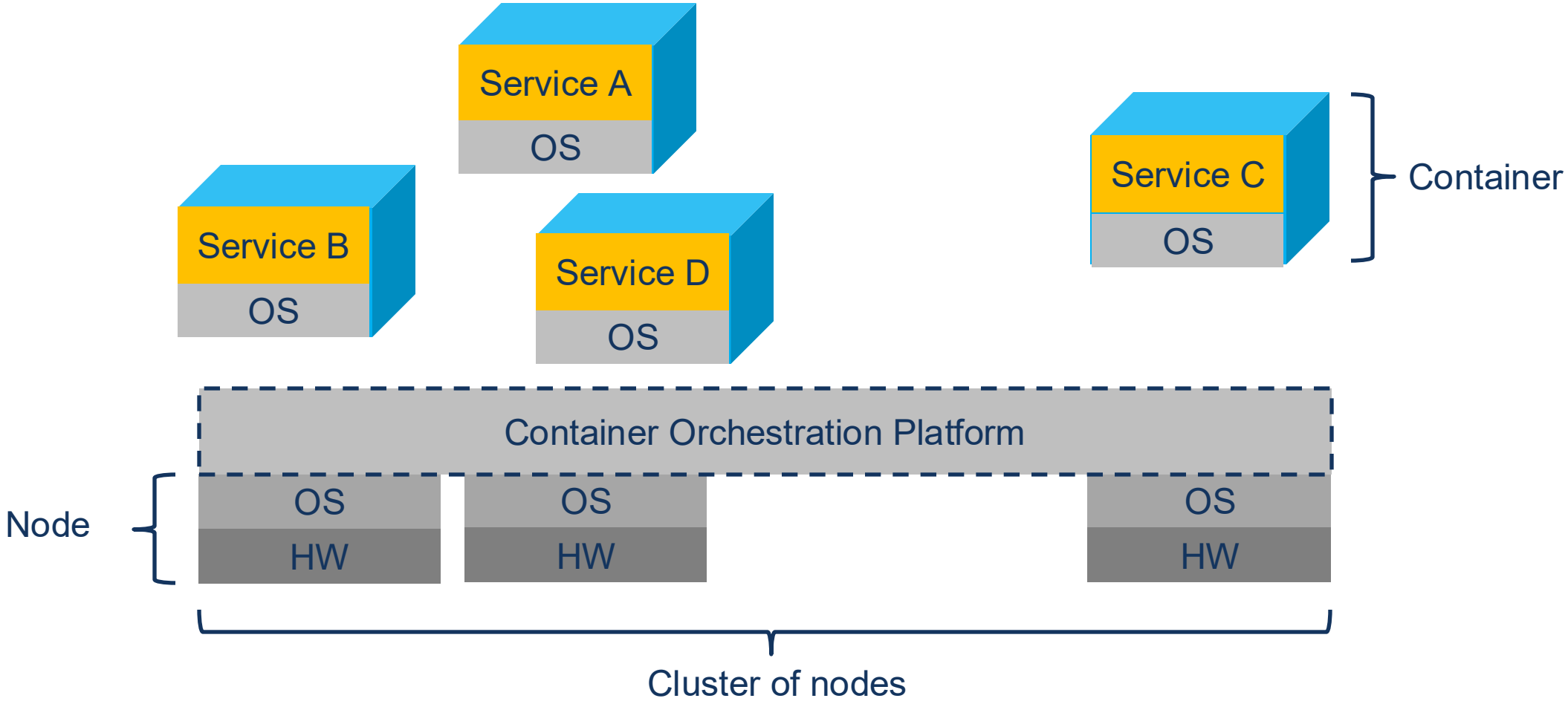
Feature	VMs	Containers	MicroVMs (Firecracker)	Unikernels	JVMs	Wasm
Isolation	Full hardware virtualisation (dedicated kernel)	OS-level isolation, shared kernel	Hardware virtualisation isolation	App + minimal OS in one image	Language-level sandbox (bytecode VM)	Runtime sandbox, memory-safe, no syscalls
Startup & Footprint	Minutes, GB scale	100 ms–seconds, MB–GB images	100–200 ms boot, ~5–10 MB	ms–100 ms boot, MB image	100 ms–seconds, tens–hundreds MB	ms startup, tens MB
Performance	Near-native, but heavier I/O	Near-native, good for JIT workloads	Near-native, low I/O overhead	Near-native, deterministic	Near-native with JIT; slower startup	10–30% slower, fast steady-state
Security	Strong, mature hardening	Kernel is shared risk	Strong isolation, small surface	Tiny surface, fewer tools	Memory-safe language, sandboxed bytecode	Memory-safe, no syscalls by default
Ecosystem	Mature (VMware, KVM, Hyper-V, EC2)	Huge (Docker, Kubernetes, OCI)	Growing (Firecracker, Cloud Hypervisor, Kata)	Niche (Unikraft, OSv, MirageOS)	Huge (JVM, GraalVM, Android ART)	Emerging (WasmEdge, Spin, Fermyon, Krustlet)
Best Fit	Legacy or multi-OS workloads	Cloud-native apps, microservices	Serverless & multi-tenant cloud	Specialized appliances, IoT	Enterprise backends, Android, polyglot apps	Edge/serverless, plugin execution
Example Use Cases	Enterprise VMs (AWS, VMware)	Netflix microservices	AWS Lambda, Fly.io, Fargate	NFV, firewalls, IoT gateways	Java/Scala services, Android apps	Cloudflare Workers, Edge compute

Trade-offs


Startup time
VMs
MicroVMs
Unikernels
Containers
Wasm
Isolation
Containers
VMs
MicroVMs
Unikernels
Wasm
Extensibility
Flexibility
Wasm
Unikernels
MicroVMs
VMs
Containers
Footprint
VMs
MicroVMs
Containers
Unikernels
Wasm
Performance
Wasm
VMs
MicroVMs
Unikernels
Containers
Worse
Better

Next Lecture

How to handle failures? How to connect, configure, scale containerised apps?



How to hide the complexity of dealing with multiple machines?


LULEÅ
UNIVERSITY
OF TECHNOLOGY